내 실기 요약 - 시험 직전 핵심

다운로드할 때 남아 있는 카드만 과목별로 모았습니다. 과목 안에서는 내가 정한 순서를 유지합니다. 왼쪽 열부터 읽고 오른쪽 열로 이어 확인하세요. 계산 예제는 남아 있는 관련 카드에 맞춰 마지막에 모았습니다.

생성 패턴

Factory Method (팩토리 메서드) | 공통 생성 인터페이스를 두고 구체 객체의 종류는 하위 클래스가 결정합니다.

Abstract Factory (추상 팩토리) | 서로 호환되는 여러 객체를 한 제품군으로 묶어 일관된 조합으로 생성합니다.

Builder (빌더) | 복잡한 객체의 생성 과정을 단계로 나누고 필요한 구성 요소를 조립합니다.

Prototype (프로토타입) | 설정이 끝난 기존 객체를 복제하여 같은 구조의 새 객체를 생성합니다.

Singleton (싱글톤) | 인스턴스를 하나만 유지하여 프로그램 전체에서 같은 객체를 공유합니다.

구별: Factory Method는 한 객체의 생성 결정을 하위 클래스에 맡기고, Abstract Factory는 관련된 여러 제품을 한 제품군으로 만듭니다.

구조 패턴

Adapter (어댑터) | 기존 인터페이스를 호출자가 기대한 형태로 바꾸어 두 객체를 연결합니다.

Bridge (브리지) | 추상화와 구현을 별도 계층으로 나누어 두 변화 축을 독립적으로 확장합니다.

Composite (컴포지트) | 개별 객체와 객체 묶음을 같은 인터페이스로 다루어 트리 구조를 처리합니다.

Decorator (데코레이터) | 기존 객체를 같은 인터페이스로 감싸 실행 중 필요한 기능을 추가합니다.

Facade (퍼사드) | 여러 하위 시스템의 복잡한 호출을 하나의 단순한 인터페이스로 제공합니다.

Flyweight (플라이웨이트) | 많은 객체의 공통 상태를 공유하여 유사 객체들의 메모리 사용을 줄입니다.

Proxy (프록시) | 실제 객체와 같은 인터페이스의 대리 객체가 접근 권한과 생성 시점을 관리합니다.

구별: Adapter는 인터페이스를 바꾸고, Decorator는 기능을 더하며, Proxy는 접근을 대신 관리합니다.

행위 패턴

Chain of Responsibility (책임 연쇄) | 여러 처리기를 연결하고 해결하지 못한 요청은 다음 처리기로 넘깁니다.

Command (커맨드) | 실행할 요청을 객체에 담아 저장하거나 취소할 수 있는 형태로 만듭니다.

Interpreter (인터프리터) | 간단한 문법을 클래스로 표현하고 문장을 해석하여 결과를 계산합니다.

Iterator (이터레이터) | 컬렉션의 내부 구조를 노출하지 않고 요소를 순차적으로 탐색합니다.

Mediator (중재자) | 객체들이 서로 직접 통신하지 않고 중재자 객체를 통해 협력하게 합니다.

Memento (메멘토) | 객체의 상태를 저장해 두었다가 나중에 이전 상태로 복원합니다.

Observer (옵저버) | 한 객체의 상태가 바뀌면 등록된 여러 구독 객체에 알려 함께 갱신합니다.

State (상태) | 현재 상태를 담당하는 객체에 맡겨 같은 요청도 상태별로 다르게 처리합니다.

Strategy (전략) | 같은 목적의 알고리즘을 독립 객체로 만들어 상황에 맞는 구현으로 교체합니다.

Template Method (템플릿 메서드) | 상위 클래스가 실행 순서를 정하고 일부 단계의 구현은 하위 클래스에 맡깁니다.

Visitor (방문자) | 객체 구조의 원소 클래스를 바꾸지 않고 새로운 연산을 추가합니다.

구별: Observer는 다른 객체들에 변화를 알리고, State는 한 객체의 현재 상태에 따라 자기 행동을 바꿉니다.

순차·검증형 모델

폭포수 모델 (Waterfall) | 요구사항부터 유지보수까지 각 단계를 순서대로 완료하는 선형 모델.

프로토타이핑 모델 (Prototyping) | 시제품을 먼저 만들어 사용자의 요구사항을 확인하고 보완하는 모델.

나선형 모델 (Spiral) | 계획·위험 분석·개발·평가를 반복하며 위험을 줄이는 모델.

V-모델 (V-Model) | 개발 단계와 대응하는 시험 단계를 연결해 검증과 확인을 강조하는 모델.

애자일·반복형 방법론

애자일 (Agile) | 계획을 고정하기보다 고객 협력과 변화 대응을 중시하는 반복 개발 방식.

XP (익스트림 프로그래밍) | 테스트 우선, 짝 프로그래밍, 지속적 통합 등 개발 실천법을 강하게 적용하는 방법론.

Scrum (스크럼) | 제품 백로그를 스프린트 단위로 개발하는 대표적인 애자일 프레임워크.

Kanban (칸반) | 업무를 시각화하고 진행 중인 작업 수를 제한해 흐름을 개선하는 방법.

FDD (기능 중심 개발) | 기능 목록을 기준으로 설계·구현을 짧게 반복하는 애자일 방법론.

Lean (린) | 낭비를 줄이고 고객에게 가치가 흐르도록 개선을 반복하는 개발 방식.

DSDM (동적 시스템 개발 방법) | 시간과 비용을 고정하고 우선순위를 조정해 정해진 기한에 결과를 내는 애자일 방법론.

요구사항 종류

기능 요구사항 (Functional) | 로그인·조회처럼 시스템이 수행할 기능과 필요한 입력·출력을 정의합니다.

비기능 요구사항 (Non-functional) | 응답 시간·보안 수준·자원 한도 등 시스템의 품질과 운영 제약을 정의합니다.

사용자 요구사항 (User) | 사용자의 업무 목표와 서비스에 기대하는 기능을 이해하기 쉬운 말로 씁니다.

시스템 요구사항 (System) | 사용자 요구를 구현하고 검증할 수 있도록 구체적인 동작과 조건으로 명세합니다.

분석·검증 절차

요구사항 도출 (Elicitation) | 인터뷰·설문·관찰·워크숍·문서 분석으로 요구를 찾아내는 활동입니다.

요구사항 분석 (Analysis) | 수집한 요구의 우선순위·충돌·의존성·실현 가능성을 검토하는 활동입니다.

요구사항 명세 (Specification) | 합의한 요구사항을 일관되고 검증 가능한 문서로 표현하는 활동입니다.

요구사항 검증 (Validation) | 정확성·완전성·일관성·추적성·검증 가능성을 확인하는 활동입니다.

추적성 관리 (Traceability) | 요구사항과 설계·코드·시험 항목의 연결 관계를 관리하는 활동입니다.

현행 시스템 분석

현행 시스템 파악 | 조직·업무·응용·데이터·기술 환경을 조사해 현재 구조와 문제를 파악합니다.

이해관계자 분석 | 프로젝트에 영향을 주거나 결과의 영향을 받는 사람과 조직을 식별합니다.

타당성 분석 (Feasibility) | 기술·경제·운영·법규 측면에서 개발 가능성과 가치를 검토합니다.

구조 다이어그램

클래스 다이어그램 (Class) | 클래스의 속성·연산과 클래스 사이의 관계를 표현합니다.

객체 다이어그램 (Object) | 특정 시점에 존재하는 객체와 속성 값, 객체 간 링크를 표현합니다.

컴포넌트 다이어그램 (Component) | 컴포넌트와 제공·요구 인터페이스 사이의 의존 관계를 표현합니다.

배치 다이어그램 (Deployment) | 실행 컴포넌트가 물리적인 노드에 어떻게 배치되는지 표현합니다.

패키지 다이어그램 (Package) | 모델 요소를 패키지로 묶고 패키지 간 의존 관계를 표현합니다.

행위·상호작용 다이어그램

유스케이스 다이어그램 (Use Case) | 액터와 시스템 기능 사이의 상호작용을 사용자 목표 중심으로 표현합니다.

시퀀스 다이어그램 (Sequence) | 시간 순서에 따른 객체 간 메시지 호출과 응답을 표현합니다.

커뮤니케이션 다이어그램 (Communication) | 객체 간 메시지와 연결 관계를 번호로 표현합니다.

활동 다이어그램 (Activity) | 업무나 연산의 흐름, 조건 분기, 병렬 처리를 표현합니다.

상태 다이어그램 (State) | 객체가 사건에 따라 상태를 바꾸는 생명주기를 표현합니다.

UML 관계

연관 (Association) | 두 객체가 서로 연결되어 정보를 주고받는 관계이며 상속과는 구별됩니다.

일반화 (Generalization) | 하위 요소가 상위의 특성을 물려받으며 상위 요소의 한 종류에 해당합니다.

의존 (Dependency) | 다른 요소를 일시적으로 사용하므로 사용 대상의 변경에 영향을 받습니다.

집합 (Aggregation) | 전체가 부분을 포함하지만 부분은 전체 없이도 독립적으로 존재할 수 있습니다.

합성 (Composition) | 전체가 부분을 강하게 소유하여 전체가 사라지면 부분도 함께 사라집니다.

실체화 (Realization) | 클래스가 인터페이스에 선언된 기능을 구현하여 정해진 계약을 충족합니다.

구별: 집합과 합성은 모두 전체·부분이지만, 부품의 독립적인 생존 여부가 핵심 차이입니다.

객체지향 핵심

추상화 (Abstraction) | 복잡한 대상에서 중요한 공통 특성만 뽑아 표현합니다.

캡슐화 (Encapsulation) | 데이터와 데이터를 처리하는 메서드를 하나로 묶고 내부 상태를 외부에서 직접 바꾸지 못하게 합니다.

상속 (Inheritance) | 상위 클래스의 속성과 동작을 하위 클래스가 물려받아 재사용합니다.

다형성 (Polymorphism) | 같은 인터페이스나 메시지에 객체 종류에 따라 다른 구현이 실행되도록 합니다.

정보 은닉 (Information Hiding) | 변경될 가능성이 높은 설계 결정과 내부 구현을 모듈 밖에 숨깁니다.

응집도

응집도는 높을수록 좋고 기능적 응집도가 가장 이상적이며 우연적 응집도가 가장 약합니다.

기능적 응집도 (Functional) | 모든 요소가 하나의 명확한 기능을 완성하기 위해 서로 밀접하게 협력합니다.

순차적 응집도 (Sequential) | 앞 작업의 출력이 다음 작업의 입력이 되어 단계 사이에 데이터가 흐릅니다.

교환적 응집도 (Communicational) | 같은 입력 데이터나 출력 결과를 다룬다는 이유로 여러 작업을 묶습니다.

절차적 응집도 (Procedural) | 데이터 흐름보다 정해진 실행 순서를 따라야 한다는 이유로 기능을 묶습니다.

시간적 응집도 (Temporal) | 기능은 달라도 초기화나 종료처럼 같은 시점에 실행된다는 이유로 모읍니다.

논리적 응집도 (Logical) | 성격이 비슷한 기능을 모으고 외부 선택값에 따라 특정 기능을 실행합니다.

우연적 응집도 (Coincidental) | 기능·데이터·실행 시점에 관련성이 없는 작업들을 편의상 한 모듈에 모읍니다.

구별: 순차적은 데이터가 단계 사이를 흐르고, 절차적은 실행 순서만 공유합니다.

결합도

결합도는 낮을수록 좋고 자료 결합도가 가장 바람직하며 내용 결합도가 가장 강합니다.

자료 결합도 (Data) | 필요한 단순 값만 인자로 전달하여 다른 모듈의 내부 구조에 덜 의존합니다.

스탬프 결합도 (Stamp) | 일부 값만 보내지 않고 구조체 전체를 전달하여 상대의 자료 구조에 의존합니다.

제어 결합도 (Control) | 상대의 처리 기능이나 실행 순서를 정하는 제어값을 전달하여 동작을 바꿉니다.

외부 결합도 (External) | 같은 외부 파일 형식이나 통신 규약·장치를 사용하여 외부 조건에 의존합니다.

공통 결합도 (Common) | 동일한 전역 데이터를 읽고 수정하여 한 모듈의 변경이 다른 모듈에 퍼집니다.

내용 결합도 (Content) | 다른 모듈의 내부 변수나 코드를 직접 참조하여 상대의 구현에 강하게 의존합니다.

구별: 자료는 필요한 값만, 스탬프는 묶인 자료 전체를 넘깁니다. 제어는 데이터가 아니라 상대의 동작을 바꿉니다.

UI 설계 원칙

가시성 (Visibility) | 현재 상태와 가능한 행동을 사용자가 쉽게 볼 수 있게 합니다.

일관성 (Consistency) | 비슷한 기능과 정보는 화면마다 같은 표현과 조작 방식을 사용합니다.

피드백 (Feedback) | 사용자의 행동 결과와 시스템 처리 상태를 즉시 알려줍니다.

오류 예방과 회복 | 실수를 미리 막고 발생한 오류는 원인과 해결 방법을 알려줍니다.

접근성 (Accessibility) | 다양한 능력과 환경의 사용자가 기능을 이용할 수 있게 합니다.

직관성 (Intuitiveness) | 화면을 처음 보는 사용자도 기능과 조작 방법을 쉽게 예상할 수 있는 정도입니다.

유효성 (Effectiveness) | 사용자가 의도한 목표를 정확하고 빠르게 달성할 수 있는 정도입니다.

학습성 (Learnability) | 초보 사용자가 짧은 시간 안에 사용법을 배우고 익숙해질 수 있는 정도입니다.

유연성 (Flexibility) | 사용자의 숙련도와 상황에 따라 다양한 방식으로 기능을 사용할 수 있는 정도입니다.

UI 설계 산출물

와이어프레임 (Wireframe) | 화면의 배치와 정보 구조를 단순한 선과 영역으로 표현한 설계 초안입니다.

목업 (Mockup) | 실제 화면과 가까운 색상·글꼴·이미지를 포함한 정적인 시각 설계입니다.

프로토타입 (Prototype) | 화면을 실제처럼 연결해 사용자 흐름과 상호작용을 검증하는 시제품입니다.

스토리보드 (Storyboard) | 화면 단위의 구성과 화면 전환, 설명을 순서대로 정리한 문서입니다.

스타일 가이드 (Style Guide) | 색상·글꼴·컴포넌트·간격·상태 등 UI 표현 규칙을 정리한 기준입니다.

아키텍처 구조

계층형 아키텍처 (Layered) | 표현·업무·데이터처럼 책임을 계층으로 나누고 인접 계층을 통해 통신합니다.

클라이언트-서버 (Client-Server) | 서비스를 요청하는 클라이언트와 요청을 처리하는 서버로 분리합니다.

MVC (Model-View-Controller) | 데이터·화면·입력 제어 책임을 Model·View·Controller로 분리합니다.

파이프-필터 (Pipe-Filter) | 처리 단계를 필터로 나누고 파이프로 중간 결과를 다음 단계에 전달합니다.

마이크로서비스 (Microservices) | 업무 기능을 독립적으로 배포·확장 가능한 작은 서비스로 나눕니다.

Master-Slave (마스터-슬레이브) | 마스터가 작업을 분배하고 슬레이브가 실제 처리를 수행하는 구조입니다.

Broker (브로커) | 브로커가 요청자와 서비스 제공자 사이에서 통신을 중개하는 구조입니다.

Peer-to-Peer (피어 투 피어) | 모든 노드가 서버와 클라이언트 역할을 함께 수행하는 분산 구조입니다.

Event Bus (이벤트 버스) | 이벤트 버스가 발생한 이벤트를 관심 있는 여러 컴포넌트에 전달하는 구조입니다.

Blackboard (블랙보드) | 공유 저장 공간에 여러 전문 모듈이 정보를 기록하고 읽으며 문제를 해결하는 구조입니다.

Interpreter (인터프리터) | 문장을 문법 구조로 표현하고 해석기가 그 구조를 실행하는 구조입니다.

인터페이스 설계

인터페이스 요구사항 | 연계 대상과 목적, 주기, 데이터, 제약 조건을 자연어로 정리한 요구사항입니다.

인터페이스 상세 설계 | 요청·응답 메시지와 호출 순서, 오류 처리, 인증 방식을 구현 가능한 수준으로 정의합니다.

데이터 매핑 (Data Mapping) | 송신 시스템의 항목과 수신 시스템의 항목, 변환 규칙을 연결합니다.

동기 통신 | 요청을 보낸 쪽이 응답을 받을 때까지 결과를 기다리는 통신 방식입니다.

비동기 통신 | 요청을 전달한 뒤 결과를 기다리지 않고 다른 작업을 계속할 수 있는 통신 방식입니다.

인터페이스 오류 처리 | 실패 원인과 재처리 가능 여부를 구분해 응답·로그·알림 규칙을 정합니다.

기업 시스템 통합

Point-to-Point | 시스템끼리 직접 연결해 데이터를 주고받는 가장 단순한 통합 방식입니다.

EAI (Enterprise Application Integration) | 기업 내부의 여러 애플리케이션을 통합해 데이터와 업무를 연결하는 방식입니다.

Hub & Spoke | 중앙 허브가 각 시스템과 연결되어 메시지 전달과 변환을 담당합니다.

ESB (Enterprise Service Bus) | 버스 형태의 표준 통신 기반에서 서비스 연결·변환·라우팅을 수행합니다.

Message Bus | 공유 메시지 채널을 통해 여러 시스템이 이벤트와 데이터를 발행·구독합니다.

Hybrid EAI (하이브리드 EAI) | 허브 중심 통합과 점대점 연결을 업무 특성에 맞게 함께 사용하는 통합 방식입니다.

웹 서비스와 데이터 교환

API (Application Programming Interface) | 다른 프로그램이 기능이나 데이터에 접근할 수 있도록 정한 호출 규칙입니다.

SOAP | XML 기반 메시지와 표준 규약을 사용하는 웹 서비스 통신 방식입니다.

WSDL (Web Services Description Language) | 웹 서비스의 기능·메시지·접근 방법을 XML로 기술하는 서비스 명세입니다.

UDDI (Universal Description, Discovery and Integration) | 웹 서비스를 등록하고 검색하기 위한 표준 디렉터리입니다.

REST | 자원을 URI로 표현하고 HTTP 메서드로 자원에 대한 행위를 표현하는 아키텍처 스타일입니다.

AJAX (Asynchronous JavaScript and XML) | 브라우저가 전체 화면을 다시 불러오지 않고 서버와 비동기 통신하는 방식입니다.

XML | 태그로 데이터 구조와 의미를 표현하는 확장 가능한 마크업 형식입니다.

JSON | 키와 값의 구조로 데이터를 간결하게 표현하는 교환 형식입니다.

구조적 분석 도구

자료 흐름도 (DFD) | 프로세스·자료 흐름·자료 저장소·단말을 이용해 자료의 이동과 변환을 표현합니다.

자료 사전 (DD) | DFD에 등장하는 자료 흐름과 저장소, 자료 항목의 의미와 구조를 정의합니다.

소단위 명세서 (Mini-Spec) | DFD의 하위 프로세스가 어떤 조건과 절차로 처리되는지 설명합니다.

HIPO (Hierarchy Input Process Output) | 입력·처리·출력을 계층 구조로 표현해 기능의 전체와 세부 관계를 보여줍니다.

CASE 도구 | 요구사항 분석과 구조적 분석·설계를 자동화하거나 지원하는 도구입니다.

설계 원리

분할과 정복 (Divide and Conquer) | 큰 문제를 서로 관리 가능한 작은 문제로 나누어 각각 해결합니다.

모듈화 (Modularity) | 시스템 기능을 독립적인 모듈 단위로 나누어 개발·재사용·시험합니다.

추상화 (Abstraction) | 불필요한 세부 구현을 생략하고 문제 해결에 필요한 관점만 표현합니다.

단계적 분해 (Stepwise Refinement) | 상위 수준의 기능을 반복해서 세부 기능으로 구체화하는 하향식 설계 전략입니다.

정보 은닉 (Information Hiding) | 변경될 가능성이 높은 자료와 절차를 모듈 내부에 숨깁니다.

컴포넌트 (Component) | 명확한 역할과 인터페이스를 가진 재사용 가능한 실행 단위입니다.

명세·분석 모델

정형 명세 (Formal) | 수학적 기호와 엄밀한 규칙으로 요구사항을 표현하는 방식입니다.

비정형 명세 (Informal) | 자연어·그림·표를 활용해 요구사항과 분석 결과를 이해하기 쉽게 표현합니다.

객체 모델링 | 객체와 클래스, 객체 사이의 관계를 찾아 정적인 구조로 표현합니다.

동적 모델링 | 시간의 흐름과 사건에 따라 객체나 시스템의 상태가 어떻게 변하는지 표현합니다.

기능 모델링 | 시스템이 입력 자료를 어떤 처리 과정으로 출력 자료로 바꾸는지 표현합니다.

UML 확장 다이어그램

복합체 구조 다이어그램 (Composite Structure) | 클래스나 컴포넌트 내부의 부품·포트·커넥터 구조를 표현합니다.

타이밍 다이어그램 (Timing) | 시간 축에 따른 객체 상태 변화와 시간 제약을 명시적으로 표현합니다.

상호작용 개요 다이어그램 (Interaction Overview) | 여러 상호작용 다이어그램 사이의 제어 흐름을 활동 흐름처럼 표현합니다.

UML 구성요소 | UML은 사물(Things), 관계(Relationship), 다이어그램(Diagram)으로 구성됩니다.

SOLID 원칙

SRP 단일 책임 원칙 | 클래스는 하나의 변경 이유와 하나의 책임만 가져야 합니다.

OCP 개방 폐쇄 원칙 | 확장에는 열려 있고 수정에는 닫혀 있어야 합니다.

LSP 리스코프 교체 원칙 | 하위 클래스는 상위 타입이 사용되는 자리에 대체되어도 상위 규약을 깨뜨리지 않아야 합니다.

ISP 인터페이스 분리 원칙 | 클라이언트가 사용하지 않는 메서드에 의존하지 않도록 작은 인터페이스로 나눕니다.

DIP 의존성 역전 원칙 | 상위 정책과 하위 구현이 구체 클래스가 아닌 추상화에 의존해야 합니다.

객체지향 구현 지표

팬인 (Fan-in) | 특정 모듈을 사용하는 상위 모듈의 수입니다.

팬아웃 (Fan-out) | 특정 모듈이 직접 사용하는 하위 모듈의 수입니다.

오버라이딩 (Overriding) | 하위 클래스가 상위 클래스의 같은 메서드를 재정의하는 방식입니다.

오버로딩 (Overloading) | 같은 클래스에서 메서드 이름은 같고 매개변수 개수나 타입이 다른 메서드를 정의합니다.

계획·일정 관리

SDLC (소프트웨어 생명주기) | 프로젝트 계획·요구 분석·설계·구현·테스트·유지보수의 전 과정을 의미합니다.

WBS (Work Breakdown Structure) | 프로젝트 목표를 수행 가능한 작업과 하위 작업으로 계층적으로 분해합니다.

PERT | 작업별 낙관치·기대치·비관치를 이용해 불확실한 프로젝트 일정을 예측합니다.

CPM (Critical Path Method) | 작업의 확정 기간과 의존 관계로 가장 오래 걸리는 임계 경로를 찾습니다.

간트 차트 (Gantt Chart) | 시간 축에 작업의 시작·종료 기간을 막대 형태로 표현합니다.

코드·노력 기반 산정

LOC (Source Lines of Code) | 예상 소스 코드 줄 수를 기준으로 노력·생산성·개발 기간을 추정합니다.

COCOMO | 원시 코드 줄 수와 프로젝트 유형을 이용해 노력과 비용을 산정하는 모델입니다.

Putnam 모델 | 소프트웨어 생명주기 전체의 노력 분포를 Rayleigh-Norden 곡선으로 추정합니다.

Effort Per Task | 생명주기의 각 단계와 세부 작업별 인월을 따로 산정해 전체 노력을 계산합니다.

기능·전문가 기반 산정

Function Point (FP) | 입력·출력·사용자 질의·내부 파일·외부 인터페이스 기능에 가중치를 주어 규모를 산정합니다.

ESTIMACS | Function Point 방식에 기반한 자동화 비용 추정 도구입니다.

전문가 감정 기법 | 경험이 있는 한 명 이상의 전문가에게 프로젝트 비용과 기간을 추정하게 합니다.

델파이 기법 | 조정자가 여러 전문가의 의견을 익명으로 반복 수렴해 합의된 추정치를 얻습니다.

아키텍처 5가지 뷰

Use-case View (유스케이스 뷰) | 사용자와 시스템의 기능 요구 및 주요 시나리오를 표현합니다.

Logical View (논리 뷰) | 클래스·객체·컴포넌트 등 시스템의 정적 구조와 책임 분할을 표현합니다.

Process View (프로세스 뷰) | 실행 중인 프로세스·스레드·통신·동시성·성능을 표현합니다.

Implementation View (구현 뷰) | 소스 코드·컴포넌트·패키지 등 구현 단위의 구성을 표현합니다.

Deployment View (배포 뷰) | 소프트웨어가 실행되는 서버·장치·네트워크의 물리적 배치를 표현합니다.

아키텍처 평가 모델

SAAM | 시나리오를 사용해 변경 용이성 등 품질 속성과 아키텍처의 영향을 분석합니다.

ATAM | 성능·가용성·보안·수정 용이성 같은 품질 속성 사이의 절충과 위험을 평가합니다.

CBAM | 아키텍처 전략의 비용과 품질 향상 효과를 함께 분석해 투자 우선순위를 정합니다.

ADR (아키텍처 결정 기록) | 중요한 아키텍처 선택과 그 배경·대안·결과를 문서로 남깁니다.

ARID (초기 설계 검토) | 초기 아키텍처의 구현 가능성과 적절성을 시나리오로 검토하는 평가 방법입니다.

비기능 요구사항

운영 요구사항 | 시스템이 운영되는 환경, 절차, 조직, 호환성, 배포와 관련된 조건입니다.

자원 요구사항 | CPU·메모리·저장 공간·네트워크 대역폭·인력과 같은 자원의 사용 한계를 정의합니다.

성능 요구사항 | 응답시간·처리량·동시 사용자·처리 주기 등 시스템의 성능 목표를 정의합니다.

품질 속성 요구사항 | 보안·신뢰성·가용성·이식성·유지보수성·사용성 등 품질 목표를 정의합니다.

아키텍처 패턴

Layered (계층형) | 시스템을 프레젠테이션·업무·데이터 같은 계층으로 나누고 인접 계층을 통해 호출합니다.

Pipe-and-Filter (파이프-필터) | 필터가 데이터를 변환하고 파이프가 필터 사이의 결과를 전달하는 처리 구조입니다.

MVC | 모델·뷰·컨트롤러의 책임을 분리하는 애플리케이션 구조입니다.

Client-Server | 서비스를 요청하는 클라이언트와 요청을 처리하는 서버로 역할을 나누는 구조입니다.

Broker (브로커) | 중개자가 서비스 등록·탐색·호출을 담당해 클라이언트와 서버의 직접 결합을 줄입니다.

XP의 다섯 가지 가치

의사소통 (Communication) | 고객과 개발자가 정보를 자주 공유해 오해를 줄입니다.

단순성 (Simplicity) | 현재 필요한 기능을 가장 단순한 설계로 구현합니다.

피드백 (Feedback) | 테스트와 고객 반응을 빠르게 받아 방향을 조정합니다.

용기 (Courage) | 문제를 드러내고 필요한 변경을 실행합니다.

존중 (Respect) | 팀원과 고객의 역할·기여·의견을 존중합니다.

XP의 개발 실천법

짝 프로그래밍 (Pair Programming) | 두 개발자가 함께 코드를 작성·검토합니다.

지속적 통합 (Continuous Integration) | 작은 변경을 자주 합치고 자동으로 검증합니다.

공동 코드 소유 (Collective Ownership) | 팀 전체가 코드의 개선 책임을 공유합니다.

작은 릴리스 (Small Releases) | 유용한 기능을 작은 단위로 자주 전달합니다.

TDD (테스트 주도 개발) | 실패하는 테스트를 먼저 쓰고 통과시킨 뒤 개선합니다.

메타포 (Metaphor) | 시스템을 설명할 공통 비유와 용어를 사용합니다.

CASE 지원 범위와 HIPO 도표

상위 CASE (Upper CASE) | 요구 분석과 설계 단계의 모델 작성을 지원합니다.

하위 CASE (Lower CASE) | 구현·테스트·유지보수 단계의 작업을 지원합니다.

가시적 도표 (Visual Table of Contents) | HIPO의 기능 계층과 모듈 간 관계를 보여줍니다.

총체적 도표 (Overview Diagram) | 기능의 입력·처리·출력을 전체 관점에서 표현합니다.

세부적 도표 (Detail Diagram) | 총체적 도표의 처리 절차를 더 자세히 표현합니다.

인월·개발 기간·PERT 계산

개발 노력 (Man-Month, 인월) | LOC를 1인당 월간 생산성으로 나눈 총 작업량입니다.

예상 개발 기간 (Month) | 총 개발 노력을 투입 인원으로 나눈 개월 수입니다.

3점 기대치 (Three-point Estimate) | 낙관치·최빈치·비관치에 가중치를 주어 기대값을 구합니다.

PERT (프로그램 평가·검토 기법) | 작업 시간의 불확실성을 3점 추정으로 반영합니다.

CPM (주 공정법) | 선후 관계를 만족하는 경로 중 소요 시간이 가장 긴 경로를 찾습니다.

CCPM (중요 사슬 프로젝트 관리) | 작업 순서뿐 아니라 자원 제약과 버퍼를 고려합니다.

COCOMO의 개발 유형과 모델

Organic (조직형) | 경험 있는 소규모 팀이 비교적 익숙한 환경에서 개발하는 유형입니다.

Semi-detached (반분리형) | 경험과 제약이 중간 수준으로 혼재한 개발 유형입니다.

Embedded (내장형) | 하드웨어·실시간·운영 제약이 강하게 결합된 유형입니다.

Basic (기본형) | 코드 규모와 개발 유형 중심으로 노력을 추정합니다.

Intermediate (중간형) | 기본 노력에 제품·인력·개발 환경 등의 비용 요인을 보정합니다.

Detailed (상세형) | 중간형을 개발 단계별 노력 배분까지 상세화합니다.

기능점수의 다섯 기능 유형

EI (외부 입력) | 경계 밖에서 들어와 내부 데이터를 유지하거나 동작을 바꾸는 기능입니다.

EO (외부 출력) | 계산·가공한 정보를 애플리케이션 경계 밖으로 내보냅니다.

EQ (외부 조회) | 입력 요청에 따라 데이터를 조회해 결과를 반환합니다.

ILF (내부 논리 파일) | 현재 애플리케이션이 유지·관리하는 논리 데이터 집합입니다.

EIF (외부 인터페이스 파일) | 다른 애플리케이션이 유지하며 현재 시스템은 참조하는 데이터입니다.

소프트웨어 설계

URL 구성 요소와 서버로 전달되는 범위

Scheme (스킴) | 자원에 접근하는 방식을 나타내며 여기서는 https입니다.

Host·Port (호스트·포트) | 서버의 이름 또는 주소와 접속할 서비스의 번호입니다.

Path·Query (경로·쿼리) | 경로는 /notes/network, 쿼리는 page=2입니다.

Fragment (프래그먼트) | # 뒤의 mask는 자원 안의 특정 부분을 가리킵니다.

소프트웨어 개발

선형 자료구조

배열 (Array) | 같은 자료형을 연속 저장하여 인덱스로 접근하며 중간 삽입에는 이동이 필요합니다.

연결 리스트 (Linked List) | 노드를 포인터로 연결하여 순서대로 접근하고 연결을 바꾸어 삽입·삭제합니다.

스택 (Stack) | 마지막에 넣은 값부터 꺼내는 LIFO 방식으로 호출 복귀나 실행 취소에 씁니다.

큐 (Queue) | 먼저 넣은 값부터 꺼내는 FIFO 방식으로 요청을 대기 순서대로 처리합니다.

우선순위 큐 (Priority Queue) | 삽입한 순서보다 지정된 우선순위를 기준으로 먼저 처리할 데이터를 꺼냅니다.

비선형 자료구조

트리 (Tree) | 하나의 루트에서 자식 노드가 계층적으로 뻗는 비선형 구조입니다.

이진 탐색 트리 (BST) | 왼쪽 서브트리에는 작은 값, 오른쪽 서브트리에는 큰 값을 두는 이진 트리입니다.

힙 (Heap) | 완전 이진 트리 형태에서 부모와 자식의 우선순위 관계를 유지하는 구조입니다.

그래프 (Graph) | 정점과 정점을 연결하는 간선으로 관계를 표현하는 구조입니다.

탐색·정렬 알고리즘

DFS (깊이 우선 탐색) | 시작 정점에서 한 경로를 끝까지 탐색한 뒤 돌아와 다른 경로를 탐색합니다.

BFS (너비 우선 탐색) | 시작 정점과 가까운 정점부터 같은 거리 순서로 탐색합니다.

이진 탐색 (Binary Search) | 정렬된 자료의 중간값과 비교해 탐색 범위를 절반씩 줄입니다.

선택 정렬 | 남은 구간에서 가장 작은 값을 골라 앞자리와 교환하는 정렬입니다.

삽입 정렬 | 원소를 하나씩 꺼내 앞쪽의 정렬된 부분에서 알맞은 위치에 끼워 넣습니다.

버블 정렬 | 인접한 두 원소를 비교·교환하며 큰 값을 뒤쪽으로 보내는 정렬입니다.

퀵 정렬 | 피벗을 기준으로 작은 값과 큰 값을 나눈 뒤 각 부분을 재귀적으로 정렬합니다.

병합 정렬 | 배열을 반씩 나눈 뒤 정렬된 두 부분을 병합하는 정렬입니다.

트리 순회 | 트리의 모든 노드를 전위·중위·후위 순서로 방문하는 방법입니다.

시간 복잡도 | 입력 크기가 커질 때 알고리즘 실행 시간이 얼마나 증가하는지 나타내는 기준입니다.

알고리즘 설계 기법 | 문제를 나누거나, 이전 결과를 저장하거나, 현재 최선의 선택을 반복하는 문제 해결 전략입니다.

연계 데이터

XML | 태그와 속성으로 계층 구조와 데이터 의미를 표현하는 문서 형식입니다.

JSON | 키와 값으로 데이터를 간결하게 표현하는 교환 형식입니다.

CSV 형식 | 쉼표 같은 구분자로 각 필드를 나누어 표현하는 텍스트 데이터 형식입니다.

고정 길이 전문 | 각 필드가 정해진 문자 위치와 길이를 차지하는 연계 데이터 형식입니다.

데이터 매핑 | 송신 시스템의 필드와 수신 시스템의 필드 및 변환 규칙을 연결하는 작업입니다.

연계 모듈과 통신

동기 통신 | 요청자가 응답을 받을 때까지 다음 처리를 기다리는 통신 방식입니다.

비동기 통신 | 요청을 메시지나 큐에 맡기고 응답을 기다리지 않은 채 다음 작업을 수행하는 방식입니다.

연계 모듈 | 송신·수신·중계·변환 기능을 역할별로 나누어 시스템 간 통신을 담당하는 모듈입니다.

메시지 큐와 브로커 | 메시지를 임시 저장하고 적절한 소비자에게 전달하는 중간 구성 요소입니다.

검증과 운영

입력 검증 | 필수값·길이·형식·범위·코드 유효성을 확인한 뒤 연계를 수행합니다.

멱등성 (Idempotency) | 같은 요청을 여러 번 처리해도 결과가 한 번 처리한 것과 같도록 만드는 성질입니다.

재시도와 보상 처리 | 일시적인 실패는 다시 시도하고 되돌릴 수 없는 부분 성공은 보상 작업으로 정리합니다.

연계 로그와 모니터링 | 요청·응답·처리 시간·상관 ID·오류를 기록해 장애를 추적하는 활동입니다.

서버 구조

MVC | Model·View·Controller로 데이터·화면·입력 제어 책임을 분리하는 구조입니다.

계층형 구조 | 표현·응용·업무·데이터 접근 계층을 나누어 의존 방향을 관리하는 구조입니다.

REST API | 자원을 URI로 표현하고 HTTP 메서드와 상태 코드로 처리 결과를 전달하는 API입니다.

CRUD | 데이터의 생성·조회·수정·삭제를 의미하는 기본 처리 기능입니다.

데이터와 실행 자원

커넥션 풀 (Connection Pool) | 데이터베이스 연결을 미리 만들어 두고 요청에 빌려주는 방식입니다.

트랜잭션 (Transaction) | 여러 데이터 변경을 하나의 작업 단위로 묶어 모두 성공하거나 모두 취소되게 합니다.

Prepared Statement | SQL과 입력값을 분리해 미리 준비된 명령으로 실행하는 방식입니다.

페이지네이션 (Pagination) | 많은 결과를 한 번에 보내지 않고 페이지나 커서 단위로 나누어 조회합니다.

예외와 상태 관리

입력값 검증 | 서버가 받은 값의 형식·범위·필수 여부를 확인하는 방어 코드입니다.

예외 처리 | 예상 가능한 실패와 예상하지 못한 오류를 구분해 처리하는 방식입니다.

세션과 토큰 | 로그인 사용자의 인증 상태를 요청 사이에서 유지하는 방법입니다.

로깅과 감사 기록 | 시스템의 동작과 주요 사용자 행위를 추적할 수 있도록 기록합니다.

보안 원칙과 접근 통제

CIA 3요소 | 기밀성·무결성·가용성으로 정보 보호의 기본 목표를 나타냅니다.

식별 (Identification) | 사용자가 자신이 누구인지 계정 이름이나 식별자로 밝히는 단계입니다.

인증 (Authentication) | 제시한 신원이 실제 본인인지 비밀번호·인증 수단 등으로 확인하는 단계입니다.

인가 (Authorization) | 인증된 주체가 특정 자원에 특정 작업을 수행할 권한이 있는지 판단합니다.

책임 추적 (Accountability) | 누가 언제 어떤 작업을 했는지 로그를 남겨 행위를 추적할 수 있게 합니다.

최소 권한과 직무 분리 | 업무 수행에 필요한 최소한의 권한만 부여하고 중요한 권한을 한 사람이나 한 계정에 집중하지 않습니다.

RBAC (역할 기반 접근 제어) | 사용자 개인이 아니라 역할에 권한을 부여하고 사용자를 역할에 배정합니다.

SSO (Single Sign-On) | 한 번의 인증으로 여러 관련 시스템을 이용할 수 있게 하는 인증 방식입니다.

소프트웨어 개발

주요 취약점

SQL Injection | 입력값이 SQL 문법으로 해석되어 조회·변경·삭제가 의도와 다르게 실행되는 취약점입니다.

XSS (Cross-Site Scripting) | 공격 스크립트가 웹 페이지에 삽입되어 다른 사용자의 브라우저에서 실행되는 취약점입니다.

CSRF (Cross-Site Request Forgery) | 로그인된 사용자의 브라우저가 공격자가 의도한 요청을 대신 보내게 하는 공격입니다.

경로 탐색 (Path Traversal) | 파일 경로 입력을 조작해 허용된 디렉터리 밖의 파일에 접근하는 취약점입니다.

명령어 삽입 (Command Injection) | 사용자 입력이 운영체제 명령의 일부로 해석되어 의도하지 않은 명령이 실행되는 취약점입니다.

버퍼 오버플로 | 프로그램이 할당된 메모리 범위를 넘어 데이터를 기록해 오류나 코드 실행을 일으키는 취약점입니다.

소프트웨어 개발

암호화와 안전한 구현

대칭키 암호화 | 암호화와 복호화에 같은 비밀키를 사용하는 방식입니다.

공개키 암호화 | 공개키와 개인키의 한 쌍을 사용해 암호화나 전자서명을 수행하는 방식입니다.

해시와 Salt | 해시는 입력을 일정한 길이의 값으로 변환하며 salt는 같은 비밀번호의 결과가 같아지는 것을 막기 위한 추가 값입니다.

TLS와 안전한 쿠키 | 전송 구간을 암호화하고 쿠키의 접근·전송 조건을 제한해 세션 탈취 위험을 줄입니다.

실행 환경 방어

Stack Guard | 스택에 방어용 Canary 값을 두고 반환 주소가 덮어써졌는지 검사하는 기법입니다.
ASLR (Address Space Layout Randomization) | 스택·힙·라이브러리의 메모리 주소를 실행할 때마다 무작위로 배치하는 방어 기법입니다.
안전한 코딩 원칙 | 입력 검증, 오류 처리, 최소 권한, 민감 정보 보호처럼 취약점이 생기기 전 예방하는 구현 원칙입니다.

테스트 수준과 유형

단위 테스트 | 함수나 클래스 등 작은 기능 단위를 분리하여 개별 동작이 올바른지 검사합니다.

통합 테스트 | 여러 모듈을 연결했을 때 데이터 전달과 인터페이스가 올바른지 검사합니다.

시스템 테스트 | 구성 요소를 합친 전체 시스템이 기능·성능 등 요구사항을 만족하는지 검사합니다.

인수 테스트 | 사용자·발주처의 관점에서 실제 업무와 계약한 요구를 충족하는지 확인합니다.

재테스트 (Retesting) | 수정 전에 실패한 테스트를 다시 실행하여 같은 오류가 해결됐는지 확인합니다.

회귀 테스트 (Regression Testing) | 변경 후 기존에 정상 동작하던 기능을 다시 검사하여 수정의 부작용을 찾아냅니다.

구별: 재테스트는 고친 결함 자체, 회귀 테스트는 변경의 영향이 퍼졌을 수 있는 다른 기능을 봅니다.

블랙박스 테스트

동치 분할 테스트 | 같은 결과가 예상되는 입력 범위를 여러 동등 클래스으로 나누고 대표값을 선택합니다.

경계값 분석 | 입력 허용 구간의 양 끝과 바로 안팎을 시험하여 경계에서 나는 오류를 찾습니다.

결정 테이블 테스트 | 조건 조합과 그에 따른 행동을 표로 만들어 규칙별 결과를 확인합니다.

원인-결과 그래프 | 입력 원인과 출력 결과 사이의 논리 관계를 그래프로 표현하여 테스트를 만듭니다.

오류 추정 (Error Guessing) | 경험과 과거 결함을 바탕으로 실수하기 쉬운 입력이나 처리 상황을 시험합니다.

페어와이즈 (Pairwise) | 매개변수 값의 모든 두 가지 조합을 포함하도록 테스트 수를 줄여 구성합니다.

화이트박스와 결함 관리

문장 커버리지 | 모든 실행 가능 문장을 한 번 이상 수행하며 판단식의 참·거짓은 따로 봅니다.

분기 커버리지 (Branch Coverage) | 각 판단식 전체가 참과 거짓이 되는 경로를 실행하여 분기별 동작을 검사합니다.

조건 커버리지 (Condition Coverage) | 복합 판단식을 구성하는 각 개별 조건의 참과 거짓이 모두 평가되도록 시험합니다.

조건/결정 커버리지 (C/DC) | 개별 조건과 전체 판단 결과에 참·거짓이 모두 나타나도록 두 기준을 함께 만족합니다.

변경 조건/결정 커버리지 (MC/DC) | 다른 조건은 고정하고 한 조건의 변화가 전체 판단 결과를 바꾸는지 확인합니다.

결함 생명주기 | 결함을 발견해 기록하고 분석·수정·재테스트·종료까지 상태로 관리합니다.

결함 심각도와 우선순위 | 심각도는 장애 영향의 크기이고 우선순위는 언제 먼저 처리할지를 나타냅니다.

테스트 오라클 | 테스트 결과가 올바른지 판단할 기준이나 기대 결과입니다.

구별: C/DC는 조건과 결정의 참·거짓을 모두 확인하고, MC/DC는 각 조건의 독립적인 영향까지 확인합니다. 단락 평가로 미실행된 조건은 구분하세요.

테스트 운영과 검증

검증과 확인 (Verification & Validation) | 검증은 개발 산출물이 정해진 명세와 절차에 맞는지, 확인은 사용자의 실제 요구를 만족하는지 평가합니다.

테스트 기반 | 테스트 케이스를 만들 때 근거로 사용하는 요구사항·설계서·코드·경험 자료입니다.

테스트 하네스 | 테스트 실행에 필요한 드라이버·스텁·테스트 데이터·도구를 묶은 환경입니다.

테스트 원칙 | 테스트는 결함의 존재를 보여줄 뿐 결함이 전혀 없음을 증명하지 못한다는 기본 원칙입니다.

회복 테스트 | 장애가 발생한 뒤 시스템과 데이터가 요구한 시간·상태로 복구되는지 확인합니다.

안전 테스트 | 비정상 접근이나 공격 상황에서 시스템이 자산을 보호하는지 확인합니다.

강도 테스트 (Stress Testing) | 정상 범위를 넘는 부하에서 시스템의 동작과 회복 능력을 확인합니다.

성능 테스트 | 응답 시간·처리량·자원 사용량 등 성능 요구사항을 확인합니다.

구조 테스트 | 프로그램 내부 구조와 논리 경로를 바탕으로 동작을 검증합니다.

병행 테스트 (Parallel Testing) | 기존 시스템과 새 시스템에 같은 입력을 주어 결과를 비교합니다.

요구사항 검토 | 요구사항 문서를 읽고 모호함·누락·충돌·검증 가능성을 확인합니다.

인스펙션 | 정해진 역할과 체크리스트를 사용해 산출물의 결함을 체계적으로 찾는 공식 검토입니다.

테스트 도구와 프레임워크 | 반복 실행과 결과 수집을 자동화하는 테스트 도구와 프레임워크입니다.

패키징 구성

패키징 명세서 | 제품의 구성 요소와 설치·운영에 필요한 정보를 정의한 문서입니다.

릴리스 노트 | 새 버전의 변경 사항과 설치·업데이트에 필요한 정보를 기록합니다.

의존성 관리 | 프로그램이 사용하는 외부 라이브러리와 실행 환경의 버전을 관리합니다.

설치 스크립트 | 프로그램과 필요한 구성 요소를 정해진 순서로 배치·설정하는 자동화입니다.

삭제 스크립트 | 프로그램 파일과 구성을 안전하게 제거하는 자동화입니다.

컨테이너·Docker | 애플리케이션과 실행 환경을 이미지로 묶어 격리된 컨테이너에서 실행하는 기술입니다.

버전과 배포

버전 관리 | 제품의 변경 이력과 호환성을 식별할 수 있도록 버전 번호를 부여하는 활동입니다.

빌드와 릴리스 | 소스와 의존성을 실행 가능한 산출물로 만들고 배포 가능한 버전으로 승인하는 과정입니다.

점진 배포와 롤백 | 일부 사용자나 환경에 먼저 배포한 뒤 이상이 없으면 범위를 넓히고 문제 시 이전 버전으로 되돌립니다.

DRM (Digital Rights Management) | 디지털 제품의 사용 권한과 복제·배포를 관리하는 기술입니다.

품질 특성

기능 적합성 | 필요한 기능을 정확하고 완전하게 제공하는 정도입니다.

성능 효율성 | 자원 사용량과 응답 시간, 처리량이 요구 수준을 만족하는 정도입니다.

사용성 (Usability) | 사용자가 제품의 기능을 쉽게 배우고 목적을 효율적으로 달성할 수 있는 품질입니다.

신뢰성 (Reliability) | 정해진 조건과 시간 동안 시스템이 요구한 기능을 안정적으로 수행하는 품질입니다.

보안성 (Security) | 허가되지 않은 접근·변경·노출로부터 정보와 기능을 보호하는 품질입니다.

유지보수성 (Maintainability) | 결함 수정과 기능 변경을 분석·구현·검증하기 쉬운 품질입니다.

이식성 (Portability) | 소프트웨어를 다른 환경으로 옮기거나 적응시키기 쉬운 품질입니다.

클린 코드와 코드 스멜 | 읽기 쉽고 중복이 적으며 한 가지 책임을 명확히 가지는 코드와, 개선이 필요한 구조적 신호를 말합니다.

정적 분석 | 프로그램을 실행하지 않고 소스 코드나 산출물을 검사하는 방법입니다.

동적 분석 | 프로그램을 실제로 실행하면서 동작과 자원 사용을 검사하는 방법입니다.

형상관리 활동

형상 식별 | 관리할 산출물을 형상 항목으로 정하고 이름·식별자·버전을 부여합니다.

형상 통제 | 형상 항목에 대한 변경을 요청·검토·승인하고 승인된 내용만 반영합니다.

형상 상태 기록 | 각 형상 항목의 현재 버전·변경 내역·승인 상태를 기록하고 보고합니다.

형상 감사 | 실제 산출물과 기록이 승인된 기준선 및 요구사항과 일치하는지 검토합니다.

베이스라인 (Baseline) | 검토와 승인을 거쳐 이후 변경의 기준이 되는 고정된 형상 상태입니다.

프로세스 모델과 형상 도구

ISO/IEC 12207 | 소프트웨어 획득·개발·운영·유지보수와 지원 프로세스를 정의한 생명주기 표준입니다.

CMMI | 조직의 소프트웨어 개발 프로세스 성숙도를 평가하고 개선하기 위한 모델입니다.

SPICE (ISO/IEC 15504) | 소프트웨어 프로세스의 수행 능력과 개선 수준을 평가하는 국제 표준입니다.

Git | 각 작업자가 전체 이력을 가진 분산 버전 관리 도구입니다.

SVN | 중앙 저장소를 중심으로 변경 이력을 관리하는 버전 관리 도구입니다.

CVS | 중앙 저장소를 중심으로 파일의 버전 이력을 관리하는 도구입니다.

형상관리 도구의 기준선 관리 | 특정 시점의 소스·문서·빌드 설정을 하나의 버전으로 고정하고 필요할 때 재현하는 활동입니다.

유지보수 유형

수정 유지보수 | 운영 중 발견한 기능 오류나 계산 실수를 고쳐 원래 요구한 동작을 회복합니다.

적응 유지보수 | 운영체제·장비·법규 등 외부 환경이 달라져도 시스템이 계속 동작하도록 수정합니다.

완전 유지보수 | 새 사용자 요구를 반영하거나 성능·사용성을 개선하여 기존 기능을 발전시킵니다.

예방 유지보수 | 앞으로 생길 결함을 줄이기 위해 코드와 구조를 정리하고 유지보수를 쉽게 합니다.

개선 기법

리팩터링 (Refactoring) | 외부 동작은 유지하면서 내부 구조와 코드 품질을 개선하는 작업입니다.

역공학 (Reverse Engineering) | 실행 파일이나 기존 코드를 분석해 설계·요구사항·구조 정보를 추출하는 작업입니다.

재공학 (Re-engineering) | 기존 시스템을 분석하고 변환·재구성해 새로운 구조나 환경으로 개선하는 과정입니다.

기술 부채 | 빠른 개발을 위해 미룬 설계·품질 작업이 미래의 추가 비용과 위험으로 쌓이는 상태입니다.

변경 관리

변경 영향 분석 | 변경이 다른 모듈·데이터·인터페이스·테스트에 미치는 영향을 확인합니다.

기술 검토와 코드 리뷰 | 변경 내용의 설계·보안·성능·가독성을 다른 개발자와 검토합니다.

릴리스와 롤백 | 변경된 버전을 운영에 적용하고 문제가 생기면 안전한 이전 상태로 되돌립니다.

변경 기록과 회고 | 무엇을 왜 바꾸었고 어떤 결과가 있었는지 기록해 다음 변경의 근거로 사용합니다.

클라우드와 인프라

IaaS | 가상 서버·스토리지·네트워크 같은 인프라를 서비스로 제공합니다.

PaaS | 애플리케이션 개발과 실행을 위한 플랫폼을 서비스로 제공합니다.

SaaS | 완성된 소프트웨어를 인터넷으로 제공합니다.

FaaS | 이벤트가 발생할 때 함수를 실행하는 서비스 모델입니다.

BaaS | 인증·데이터베이스·파일 저장 같은 백엔드 기능을 서비스로 제공합니다.

DaaS | 데스크톱 환경 또는 데이터를 서비스로 제공하는 모델에 쓰이는 약어입니다.

RAID | 여러 디스크를 하나의 논리 디스크처럼 구성해 성능·용량·장애 허용성을 높이는 기술입니다.

PaaS-TA | 애플리케이션 개발·배포에 필요한 환경을 클라우드 방식으로 제공하는 국내 플랫폼 기술입니다.

연계·운영 기술

FEP (Front-End Processor) | 입력 데이터를 본 시스템이 처리하기 전에 수집·변환·검사하는 전처리 장치 또는 프로그램입니다.

MQTT | 저대역폭 환경의 장치 사이에서 발행·구독 방식으로 메시지를 전달하는 경량 프로토콜입니다.

SOA (Service-Oriented Architecture) | 업무 기능을 독립적인 서비스로 제공하고 표준 인터페이스로 조합하는 아키텍처입니다.

nmap | 네트워크 호스트와 포트, 서비스 정보를 탐색해 노출된 네트워크 자원을 확인하는 도구입니다.

Tripwire | 파일과 디렉터리의 기준 상태를 저장하고 이후 변경을 감지하는 무결성 점검 도구입니다.

테스트 케이스 구성

테스트 조건 | 테스트를 시작하기 전에 만족해야 할 상태와 전제를 정의합니다.

테스트 데이터 | 테스트 실행에 입력할 값과 경계·오류 사례를 준비합니다.

예상 결과 | 정해진 입력에 대해 시스템이 보여야 하는 구체적인 결과입니다.

판정 기준 | 실제 결과가 예상 결과와 얼마나 일치해야 성공인지 판단하는 규칙입니다.

실행 절차 | 테스트를 같은 방식으로 다시 수행할 수 있도록 작업 순서를 기록합니다.

실행 환경 | 테스트에 사용한 운영체제·브라우저·DB·네트워크 설정과 버전을 기록합니다.

테스트 오라클 | 실행 결과가 올바른지 판단하는 기준 또는 자동 판정 방법입니다.

커버리지와 통합 테스트

문장 커버리지 | 실행 가능한 각 문장이 테스트 중 한 번 이상 실행되었는지 측정합니다.

분기 커버리지 | 각 분기 판단의 참·거짓 결과를 모두 실행했는지 측정합니다.

조건 커버리지 | 복합 조건을 구성하는 각 조건의 참·거짓을 모두 확인합니다.

다중 조건 커버리지 | 복합 조건을 이루는 조건들의 가능한 참·거짓 조합을 시험합니다.

기본 경로 검사 | 제어 흐름 그래프에서 독립적인 실행 경로를 찾아 테스트합니다.

스텁 (Stub) | 테스트 중 아직 준비되지 않은 하위 호출 모듈을 대신하는 가짜 모듈입니다.

드라이버 (Driver) | 테스트 대상인 하위 모듈을 호출하는 상위 모듈을 대신하는 가짜 모듈입니다.

EAI 통합 방식 | Point-to-Point, Hub & Spoke, Message Bus, Hybrid는 시스템 사이의 연결 구조를 구분합니다.

CMMI와 리뷰

CMMI 5단계 | 초기(Initial)·관리(Managed)·정의(Defined)·정량적 관리(Quantitatively Managed)·최적화(Optimizing) 단계로 프로세스 성숙도를 구분합니다.

관리 검토 | 일정·비용·자원·위험 등 프로젝트 관리 상태를 확인합니다.

기술 검토 | 설계·구현·성능·보안 등 기술적 적합성과 결함을 확인합니다.

동료 검토 (Peer Review) | 동료 개발자가 산출물을 읽고 오류나 개선점을 제안하는 검토입니다.

워크스루 (Walkthrough) | 작성자가 산출물을 설명하며 참여자와 함께 내용을 따라가며 검토합니다.

인스펙션 (Inspection) | 정해진 역할·절차·체크리스트에 따라 결함을 체계적으로 찾는 공식 검토입니다.

나쁜 코드와 개선

Spaghetti Code (스파게티 코드) | 제어 흐름과 모듈 관계가 복잡하게 얽혀 이해와 변경이 어려운 코드입니다.

중복 코드 | 동일한 로직이 여러 위치에 반복된 상태입니다.

유사 코드 | 형태는 비슷하지만 세부 규칙이 조금씩 다른 코드가 여러 곳에 있는 상태입니다.

불명확한 이름 | 변수나 함수 이름만으로 역할과 의도를 알기 어려운 상태입니다.

매직 넘버 | 의미 설명 없이 숫자 상수가 코드에 직접 쓰인 상태입니다.

거대한 함수와 책임 혼합 | 한 함수나 클래스가 입력 검증·DB 접근·화면 출력·알림까지 모두 담당하는 상태입니다.

처리량·응답 시간·경과 시간·사용률

처리량 (Throughput) | 단위 시간 동안 완료한 작업이나 요청의 양입니다.

응답 시간 (Response Time) | 요청한 시점부터 정한 응답을 받는 시점까지 걸린 시간입니다.

경과 시간 (Turnaround Time) | 작업을 의뢰한 시점부터 처리가 완료될 때까지의 시간입니다.

자원 사용률 (Resource Utilization) | 측정 구간의 CPU·메모리·회선 등 자원 사용 비율입니다.

보안 개발 방법론

Secure SDLC (보안 개발 생명주기) | 개발 전 과정에 보안 요구와 검증 활동을 통합합니다.

MS SDL (마이크로소프트 보안 개발 생명주기) | Microsoft가 정립한 개발 과정의 보안·프라이버시 관리 방식입니다.

Seven Touchpoints (세븐 터치포인츠) | 검증된 보안 실천 활동을 개발의 주요 접점에 결합합니다.

CLASP (경량 애플리케이션 보안 프로세스) | 역할과 활동을 중심으로 기존 개발 과정에 보안을 적용합니다.

배치 유형과 스케줄러

이벤트 배치 (Event-driven Batch) | 사전에 정한 사건이나 조건이 발생하면 실행합니다.

온디맨드 배치 (On-demand Batch) | 사용자나 운영자의 명시적인 요청으로 실행합니다.

정기 배치 (Scheduled Batch) | 정해진 시각이나 주기에 맞추어 반복 실행합니다.

Spring Batch (스프링 배치) | 대량 데이터의 읽기·처리·쓰기와 재시작을 지원하는 프레임워크입니다.

Quartz (쿼츠) | Job과 Trigger를 분리해 실행 시점을 관리하는 스케줄러입니다.

cron 표현식 (크론 표현식) | 정해진 필드로 반복 실행 시각을 표현합니다.

소프트웨어 개발

테스트 더블의 다섯 유형

Dummy (더미) | 매개변수 자리를 채우지만 시험에서는 사용하지 않는 객체입니다.

Stub (스텁) | 호출 시 미리 정한 응답을 제공하는 대역입니다.

Spy (스파이) | 호출 횟수와 인자 등 사용 내역을 기록하는 대역입니다.

Mock (목) | 예상한 호출·인자·순서 등의 상호작용을 검증하는 대역입니다.

Fake (페이크) | 실제로 동작하지만 시험에 맞게 단순화한 구현입니다.

테스트 오라클의 네 유형

참 오라클 (True Oracle) | 모든 시험 입력에 대해 정확한 기대 결과를 제공할 수 있습니다.

샘플링 오라클 (Sampling Oracle) | 선정한 일부 입력에 대해서만 정확한 기대 결과를 압니다.

휴리스틱 오라클 (Heuristic Oracle) | 확정 결과와 경험적 규칙·추정을 함께 활용합니다.

일관성 검사 오라클 (Consistent Oracle) | 기존 결과나 대응하는 실행 결과와의 일관성을 비교합니다.

소프트웨어 개발

테스트의 일곱 기본 원칙

결함의 존재 (Presence of Defects) | 테스트로 결함을 찾을 수 있지만 결함이 없음을 증명할 수는 없습니다.

완벽한 시험은 불가능 (Exhaustive Testing) | 현실적인 대부분의 시스템에서 모든 입력과 경로를 시험할 수 없습니다.

조기 시험 (Early Testing) | 개발 초기에 결함을 찾으면 재작업을 줄일 수 있습니다.

결함 집중 (Defect Clustering) | 일부 영역에 많은 결함이 집중되는 경향이 있습니다.

살충제 패러독스 (Pesticide Paradox) | 같은 시험만 반복하면 새로운 결함을 찾기 어려워집니다.

정황 의존 (Context Dependency) | 제품의 목적과 환경에 따라 필요한 시험이 달라집니다.

오류 부재의 궤변 (Absence-of-errors Fallacy) | 결함이 적어도 사용자 요구를 충족하지 못하면 성공한 제품이 아닙니다.

상태·시나리오 기반 테스트

상태 전이 시험 (State Transition Testing) | 현재 상태와 이벤트에 따른 다음 상태·동작을 시험합니다.

유스케이스 시험 (Use Case Testing) | 사용자의 목표를 달성하는 기본·대안·예외 시나리오를 시험합니다.

분류 트리 시험 (Classification Tree Testing) | 입력 특성을 분류하고 각 분류의 값을 조합해 시험을 설계합니다.

비교 시험 (Comparison Testing) | 같은 입력을 여러 구현에 주어 결과 차이를 확인합니다.

DRM 구성요소와 역할

콘텐츠 제공자 (Content Provider) | 저작권을 가진 콘텐츠를 제공하는 주체입니다.

패키저 (Packager) | 콘텐츠를 암호화하고 메타데이터와 함께 배포 단위로 묶습니다.

콘텐츠 분배자 (Content Distributor) | 보호된 콘텐츠를 사용자에게 유통합니다.

클리어링 하우스 (Clearing House) | 키·라이선스 발급과 권한 관련 관리를 수행합니다.

DRM 컨트롤러 (DRM Controller) | 사용자 환경에서 콘텐츠의 이용 권한을 검사·통제합니다.

보안 컨테이너 (Security Container) | 콘텐츠와 보호 정보를 안전한 유통 단위로 담습니다.

콘텐츠 소비자 (Content Consumer) | 정해진 이용 권한 아래 콘텐츠를 사용하는 주체입니다.

콘텐츠 식별·추적·패키징

워터마킹 (Digital Watermarking) | 콘텐츠에 저작권·소유권 식별 정보를 삽입합니다.

핑거프린팅 (Digital Fingerprinting) | 구매자나 배포본별 정보를 삽입해 유출 경로를 추적합니다.

DOI (디지털 객체 식별자) | 디지털 객체를 지속적으로 식별하는 식별자입니다.

권리 표현 (Rights Expression) | 사용 기간·횟수 등 허용 조건을 기계가 처리할 형태로 표현합니다.

사전 패키징 (Pre-packaging) | 요청이 오기 전에 보호된 배포 콘텐츠를 미리 생성합니다.

즉시 패키징 (On-the-fly Packaging) | 사용자의 요청 시점에 필요한 보호·배포 형식으로 포장합니다.

조건·분기·MC/DC의 참거짓 계산

조건 커버리지 사례 | (A,B)=(참,거짓), (거짓,참)이면 A와 B는 각각 참·거짓을 갖습니다.

분기 커버리지 사례 | (참,참), (거짓,거짓)이면 D의 참·거짓 결과가 모두 존재합니다.

MC/DC의 독립 영향 | (거짓,거짓), (참,거짓), (거짓,참)을 사용하면 각 조건의 영향을 분리할 수 있습니다.

핵심 용어

데이터베이스 (Database) | 여러 사용자가 공동으로 이용하도록 통합하고 저장한 관련 데이터의 집합입니다.

DBMS (Database Management System) | 데이터를 정의·조작·제어하고 무결성·보안·동시성·복구를 관리하는 소프트웨어입니다.

스키마 (Schema) | 테이블과 속성·관계·제약 조건을 정의하며 현재 값과 구별되는 설계 명세입니다.

인스턴스 (Instance) | 특정 시점에 실제로 저장된 데이터의 상태로 행의 추가·수정·삭제에 따라 바뀝니다.

구별: 회원(회원번호, 이름, 등급)의 구조는 스키마이고 현재 저장된 회원 행들은 인스턴스입니다. 회원을 추가하면 인스턴스는 바뀌지만 구조가 같으면 스키마는 유지됩니다.

데이터베이스 구축

3단계 스키마

외부 스키마 (External Schema) | 개별 사용자나 응용 프로그램에 필요한 데이터만 보여 주는 부분 논리 구조입니다.

개념 스키마 (Conceptual Schema) | 조직 전체의 데이터와 관계·제약을 통합한 논리 구조로 사용자 관점을 조정합니다.

내부 스키마 (Internal Schema) | 파일 배치·레코드 형식·인덱스 등 저장 장치에 데이터를 배치하는 물리 구조입니다.

논리적 데이터 독립성 (Logical Data Independence) | 개념 스키마를 바꾸어도 외부 스키마와 응용 프로그램이 유지되도록 분리합니다.

물리적 데이터 독립성 (Physical Data Independence) | 내부 저장 구조를 바꾸어도 개념 스키마와 응용 프로그램이 유지되도록 분리합니다.

구별: 외부→개념 사이의 논리적 독립성과 개념→내부 사이의 물리적 독립성을 구분하세요. 인덱스 추가나 파일 배치 변경은 물리적 독립성의 예입니다.

관계형 구성 요소

릴레이션 (Relation) | 행과 열로 표현하는 데이터 집합으로 표 전체이며 이론상 중복 튜플을 허용하지 않습니다.

튜플 (Tuple) | 릴레이션의 한 행인 레코드로 하나의 대상을 나타내며 여러 속성 값을 함께 담습니다.

속성 (Attribute) | 릴레이션의 한 열인 필드로 대상의 특성을 나타내며 이름·자료형·도메인을 가집니다.

도메인 (Domain) | 한 속성에 허용되는 원자값의 집합으로 현재 저장된 값들의 목록과는 다릅니다.

카디널리티 (Cardinality) | 기수라고도 하며 릴레이션의 튜플 개수로 제목 행을 제외하고 데이터 행만 셉니다.

차수 (Degree) | 릴레이션의 속성 개수인 열 수로 데이터 행이 늘어도 구조가 같으면 유지됩니다.

구별: 회원(회원번호, 이름, 등급)에 회원 4명이 저장돼 있으면 카디널리티=4, 차수=3입니다. 등급의 허용 값이 일반·우수·VIP라면 그 집합이 도메인입니다. 모든 행을 삭제해도 구조가 같으면 차수는 3입니다. ER 관계의 카디널리티는 1:N 등 연결 수를 뜻합니다.

키의 종류

슈퍼키 (Super Key) | 행을 유일하게 식별하는 속성 집합으로 불필요한 속성이 포함되어도 됩니다.

후보키 (Candidate Key) | 행을 유일하게 식별하며 속성 하나라도 제거하면 유일성이 깨지는 최소 집합입니다.

기본키 (Primary Key) | 후보키에서 행의 대표 식별자로 선택하며 중복값과 NULL을 허용하지 않습니다.

외래키 (Foreign Key) | 다른 테이블의 기본키나 유일키를 참조하여 테이블 사이의 데이터 관계를 연결합니다.

구별: 후보키는 유일성+최소성을 모두 만족합니다. 기본키는 후보키 가운데 하나이며 NULL이 될 수 없습니다.

무결성 제약

개체 무결성 (Entity Integrity) | 각 행을 유일하게 식별하도록 기본키에 중복값이나 NULL이 들어가지 못합니다.

참조 무결성 (Referential Integrity) | NULL이 아닌 외래키 값이 부모 키에 존재하도록 하여 잘못된 참조를 막습니다.

도메인 무결성 (Domain Integrity) | 열의 값이 정해진 자료형·범위·허용 목록을 만족하도록 잘못된 입력을 막습니다.

ER 구성 요소

엔터티 (Entity) | 업무에서 독립적으로 관리하는 대상으로 고객·주문처럼 공통 속성의 개체를 묶습니다.

속성 (Attribute) | 엔터티가 가진 성질이나 상태를 표현하며 이름·주소처럼 대상의 특징을 담습니다.

관계 (Relationship) | 고객이 주문한다처럼 엔터티 사이에 맺어지는 업무상의 연관이나 사건을 나타냅니다.

식별자 (Identifier) | 엔터티의 각 인스턴스를 유일하게 구별하는 하나의 속성 또는 속성 조합입니다.

구별: Relation은 관계형 모델의 표이고 Relationship은 ER 모델에서 개체 사이의 연관입니다. 같은 "관계"라는 번역만 보고 혼동하지 마세요.

관계의 수와 참여

1:1 관계 | 양쪽의 한 개체가 상대편의 최대 한 개체와 연결되며 필수 여부는 따로 정합니다.

1:N 관계 | 한쪽의 한 개체가 상대편 여러 개체와 연결되며 일반적으로 N쪽에 외래키를 둡니다.

N:M 관계 | 양쪽 개체가 서로 여러 개체와 연결되므로 관계를 저장할 교차 테이블이 필요합니다.

참여 제약 (Participation) | 모든 개체가 관계에 반드시 참여해야 하는지 아니면 참여하지 않아도 되는지 정합니다.

구별: 고객 1명은 주문 여러 건과 연결되므로 고객:주문은 1:N입니다. 고객이 주문을 아직 안 했어도 된다면 선택 참여입니다. 여기의 카디널리티는 표의 현재 행 수가 아니라 관계의 연결 수를 뜻합니다.

데이터베이스 구축

ERD의 관계형 변환

강한 엔터티 변환 | 독립적으로 식별 가능한 엔터티를 하나의 릴레이션으로 변환합니다.

1:N 관계 변환 | 1쪽의 기본키를 N쪽 릴레이션의 외래키로 전파합니다.

N:M 관계 변환 | 두 엔터티의 기본키를 외래키로 포함하는 교차 릴레이션을 만듭니다.

약한 엔터티 변환 | 소유 엔터티의 식별자와 자신의 부분 키를 결합해 약한 엔터티를 식별합니다.

함수 종속과 이상

함수 종속 (Functional Dependency) | 릴레이션의 속성 X의 값이 속성 Y의 값을 유일하게 결정하면 X→Y로 표현합니다.

완전 함수 종속 | 복합키 전체에 종속되고 키의 일부를 제거하면 더 이상 결정되지 않는 종속입니다.

부분 함수 종속 | 복합키의 일부 속성만으로도 비키 속성이 결정되는 종속입니다.

이행 함수 종속 | X→Y이고 Y→Z일 때 X→Z가 성립하며 비키 속성이 다른 비키 속성을 결정하는 종속입니다.

정규형 단계

제1정규형 (1NF) | 각 속성에 원자값 하나만 저장하여 반복되는 값이나 다중값 속성을 없앱니다.

제2정규형 (2NF) | 1NF에서 복합 후보키의 일부에만 종속된 속성을 분리하여 부분 종속을 없앱니다.

제3정규형 (3NF) | 2NF에서 키 아닌 속성을 통해 다른 속성이 키에 의존하는 이행 종속을 없앱니다.

BCNF | 모든 비자명한 함수 종속의 결정자가 슈퍼키가 되도록 하여 3NF를 강화합니다.

제4정규형 (4NF) | 독립적인 여러 다중값 사실을 별도 테이블로 분리하여 불필요한 조합을 없앱니다.

제5정규형 (5NF) | 분해 후 다시 조인해도 원래 결과를 유지하도록 비자명한 조인 종속을 제거합니다.

구별: 2NF는 부분 함수 종속, 3NF는 이행 함수 종속, BCNF는 슈퍼키가 아닌 결정자를 구별하는 것이 핵심입니다.

분해와 이상 현상

삽입 이상 (Insertion Anomaly) | 새 데이터를 삽입할 때 불필요한 정보까지 함께 입력해야 하거나 입력할 수 없는 문제입니다.

삭제 이상 (Deletion Anomaly) | 한 데이터를 삭제했을 뿐인데 보존해야 할 다른 정보까지 함께 사라지는 문제입니다.

갱신 이상 (Update Anomaly) | 중복 저장된 같은 정보를 모두 수정하지 않아 값이 서로 달라지는 문제입니다.

무손실 분해 (Lossless Decomposition) | 분해한 릴레이션을 자연 조인했을 때 원래 릴레이션의 정보를 정확히 복원할 수 있는 성질입니다.

기본 관계대수 연산

셀렉션 (Selection, σ) | 조건을 만족하는 튜플만 선택하는 연산입니다.

프로젝션 (Projection, π) | 필요한 속성만 선택해 새로운 릴레이션을 만드는 연산입니다.

합집합 (Union, ∪) | 두 릴레이션 중 적어도 한쪽에 속한 튜플을 중복 없이 모읍니다.

교집합 (Intersection, ∩) | 두 릴레이션에 모두 속한 튜플만 선택합니다.

차집합 (Difference, -) | 첫 번째 릴레이션에는 있지만 두 번째 릴레이션에는 없는 튜플을 구합니다.

카티션 프로덕트 (×) | 두 릴레이션의 모든 튜플 조합을 만드는 연산입니다.

연결 연산

세타 조인 (θ-Join) | 두 릴레이션을 조건식에 따라 결합하는 조인입니다.

동등 조인 (Equi-Join) | 조인 조건으로 등호를 사용하는 세타 조인입니다.

자연 조인 (Natural Join) | 두 릴레이션의 같은 이름·도메인을 가진 속성을 기준으로 동등 조인하고 중복 속성을 제거합니다.

외부 조인 (Outer Join) | 조인 조건을 만족하지 않는 튜플도 보존하는 조인입니다.

디비전과 해석

디비전 (Division, ÷) | 한 릴레이션의 모든 튜플과 관련된 튜플을 다른 릴레이션에서 찾는 연산입니다.

관계대수 | 원하는 결과를 얻기 위해 어떤 연산을 어떤 순서로 수행할지 표현하는 절차적 언어입니다.

관계해석 | 어떤 튜플이 결과에 포함되어야 하는지를 논리식으로 표현하는 비절차적 언어입니다.

조회와 조건

SELECT | 테이블에서 조회할 열과 계산식을 지정하는 SQL 문입니다.

WHERE | 개별 행이 만족해야 하는 검색 조건을 지정합니다.

ORDER BY | 조회 결과를 하나 이상의 기준으로 정렬합니다.

LIKE | 문자열이 지정한 패턴에 맞는지 검사합니다.

IN | 값이 지정한 목록 또는 서브쿼리 결과에 속하는지 검사합니다.

BETWEEN | 값이 두 경계 사이에 있는지 검사하며 양 끝 경계도 포함합니다.

집계와 그룹화

COUNT | 행 또는 NULL이 아닌 특정 열의 개수를 세는 집계 함수입니다.

SUM | 지정한 숫자 열의 합계를 계산합니다.

AVG | 지정한 숫자 열의 평균을 계산합니다.

MAX | 지정한 열에서 가장 큰 값을 구합니다.

MIN | 지정한 열에서 가장 작은 값을 구합니다.

GROUP BY | 같은 값을 가진 행을 그룹으로 묶어 집계 결과를 계산합니다.

HAVING | GROUP BY로 만들어진 그룹이 만족해야 하는 조건을 지정합니다.

NULL과 표현식

NULL | 없거나 아직 알 수 없는 값의 상태로 일반 비교 대신 IS NULL로 확인합니다.

CASE | 조건에 따라 서로 다른 값을 반환하며 등급 표시나 조건별 계산에 사용합니다.

COALESCE·NVL | NULL을 대체할 값을 반환하며 COALESCE는 첫 번째 NULL 아닌 값을 선택합니다.

문자 함수 | 문자열 일부를 추출하거나 길이와 대소문자를 바꾸는 등 문자 데이터를 가공합니다.

날짜 함수 | 날짜의 구성 요소를 추출하거나 기간을 더하고 차이를 구해 날짜 데이터를 가공합니다.

구별: 값 = NULL과 NULL = NULL은 TRUE가 아닌 UNKNOWN입니다. COUNT(*)는 전체 행, COUNT(열)은 그 열이 NULL 아닌 행을 셉니다. 빈 문자열은 보통 NULL과 다르지만 Oracle은 빈 문자열을 NULL로 취급합니다.

조인 유형

INNER JOIN | 양쪽 테이블에서 조건이 일치하는 행만 결합하고 짝이 없는 행은 제외합니다.

LEFT OUTER JOIN | 왼쪽의 모든 행을 남기고 오른쪽에 짝이 없는 행의 열은 NULL로 채웁니다.

SELF JOIN | 같은 테이블에 별칭을 두 개 붙여 직원과 관리자처럼 내부의 행들을 연결합니다.

CROSS JOIN | 양쪽 테이블의 모든 행을 조합하므로 결과 행 수는 두 테이블 행 수의 곱입니다.

서브쿼리

단일행 서브쿼리 | 한 행을 반환하며 단일값 비교에서는 그 값을 일반 비교 연산자로 비교합니다.

다중행 서브쿼리 | 여러 행을 반환하므로 IN·ANY·ALL 등으로 여러 결과값과 비교합니다.

상관 서브쿼리 | 바깥 쿼리의 현재 행을 참조하여 바깥 행의 값에 따라 내부 결과가 달라집니다.

EXISTS | 조회 결과에 행이 하나라도 있는지 판단하며 반환한 열의 값 자체는 보지 않습니다.

구별: EXISTS는 상관 서브쿼리와 함께 쓰일 수 있어 서로 배타적인 분류가 아닙니다.

집합 연산

UNION | 두 조회 결과를 합치고 동일한 행은 한 번만 남겨 전체 결과의 중복을 없앱니다.

UNION ALL | 두 조회 결과를 합치되 같은 행이 여러 번 나와도 제거하지 않고 모두 유지합니다.

INTERSECT | 양쪽 조회 결과에 공통으로 있는 행만 선택하고 한쪽에만 있는 행은 제외합니다.

EXCEPT·MINUS | 첫 조회에는 있지만 두 번째 조회에는 없는 행만 남겨 차집합 결과를 구합니다.

SQL 분류

DDL (Data Definition Language) | CREATE·ALTER·DROP으로 테이블 등의 구조를 생성·변경·삭제합니다.

DML (Data Manipulation Language) | 데이터를 SELECT로 조회하거나 INSERT·UPDATE 등으로 추가·수정합니다.

DCL (Data Control Language) | GRANT·REVOKE로 사용자 접근 권한을 부여하거나 기존 권한을 회수합니다.

TCL (Transaction Control Language) | COMMIT·ROLLBACK으로 트랜잭션의 변경 결과를 확정하거나 취소합니다.

제약 조건과 객체

제약 조건 (Constraint) | 데이터가 지켜야 하는 규칙을 DBMS가 검사하도록 정의한 조건입니다.

뷰 (View) | 하나 이상의 테이블에 대한 SELECT 결과를 가상 테이블처럼 제공하는 객체입니다.

인덱스 (Index) | 검색 조건과 정렬에 사용할 수 있는 별도의 접근 경로입니다.

저장 프로시저 (Procedure) | DB에 저장된 SQL 명령과 절차를 명시적으로 호출해 실행합니다.

저장 함수 (Function) | 입력값을 받아 계산하거나 조회한 결과를 반환하는 DB 객체입니다.

트리거 (Trigger) | 테이블의 INSERT·UPDATE·DELETE 같은 사건이 발생할 때 자동으로 실행되는 DB 객체입니다.

트랜잭션과 ACID

원자성 (Atomicity) | 트랜잭션의 모든 연산이 성공하거나 모두 취소되어야 한다는 성질입니다.

일관성 (Consistency) | 트랜잭션 전후에 데이터베이스가 정의된 무결성 제약과 규칙을 만족해야 한다는 성질입니다.

격리성 (Isolation) | 동시에 실행되는 트랜잭션이 서로의 중간 결과를 보지 못하도록 하는 성질입니다.

지속성 (Durability) | 커밋이 완료된 트랜잭션의 결과가 장애 후에도 보존되는 성질입니다.

동시성 제어

공유 로크 (S Lock) | 여러 트랜잭션이 같은 데이터를 동시에 읽을 수 있도록 허용하는 읽기용 로크입니다.

배타 로크 (X Lock) | 한 트랜잭션이 데이터를 변경할 때 다른 트랜잭션의 공유·배타 로크 획득을 막는 쓰기용 로크입니다.

2단계 로킹 (2PL) | 트랜잭션이 로크를 얻는 확장 단계와 로크를 해제하는 축소 단계를 나누는 규칙입니다.

직렬성 (Serializability) | 동시에 실행된 스케줄의 결과가 어떤 직렬 실행과 동일한 상태를 만드는 성질입니다.

교착 상태 (Deadlock) | 서로가 가진 로크를 기다리며 두 개 이상의 트랜잭션이 영원히 진행하지 못하는 상태입니다.

장애 회복

로그 기반 회복 | 트랜잭션의 변경 전·후 값을 로그에 기록해 장애 후 데이터 상태를 복원합니다.

UNDO | 완료되지 않은 트랜잭션의 변경을 이전 값으로 되돌리는 회복 작업입니다.

REDO | 커밋은 완료됐지만 데이터 페이지에 반영되지 않은 변경을 다시 실행하는 회복 작업입니다.

체크포인트 (Checkpoint) | 특정 시점의 데이터와 로그 상태를 기록해 장애 회복 시 검사할 범위를 줄이는 지점입니다.

데이터베이스 구축

인덱스와 저장 구조

인덱스 (Index) | 테이블의 특정 열을 기준으로 원하는 행을 빠르게 찾도록 만든 접근 구조입니다.

선택도 (Selectivity) | 전체 행 중 특정 조건에 의해 선택되는 행의 비율 또는 구분 정도입니다.

클러스터링 | 관련된 행을 물리적으로 가까운 위치에 저장하거나 특정 키 순서로 배치하는 방식입니다.

저장 레코드 설계 | 테이블 행의 필드 배치와 자료형, 고정·가변 길이 저장 방식을 정하는 작업입니다.

파티션과 분산

범위 파티션 (Range) | 파티션 키의 연속된 값 구간을 기준으로 행을 나눕니다.

목록 파티션 (List) | 파티션 키가 미리 지정된 값 목록 중 어디에 속하는지에 따라 행을 나눕니다.

해시 파티션 (Hash) | 파티션 키에 해시 함수를 적용한 결과로 행을 분산합니다.

파티션 프루닝 | 조건에 맞지 않는 파티션을 조회 대상에서 제외하는 최적화입니다.

분산 데이터베이스 | 데이터를 여러 네트워크 노드에 분산 저장하면서 사용자에게 하나의 데이터베이스처럼 제공하는 시스템입니다.

데이터 복제 (Replication) | 같은 데이터를 여러 노드에 복사해 보관합니다.

데이터 분할 (Sharding) | 데이터를 특정 기준에 따라 여러 노드에 나누어 저장합니다.

물리 설계 점검

반정규화 (Denormalization) | 조회 성능이나 편의를 위해 정규화된 테이블을 의도적으로 통합하거나 중복을 허용하는 설계입니다.

물리적 무결성 점검 | 저장 구조 변경 후에도 키·참조·범위 제약과 애플리케이션의 가정이 유지되는지 확인합니다.

성능 모니터링 | 실행 계획·응답 시간·I/O·캐시 적중률·락 대기 등 운영 지표를 관찰하는 활동입니다.

인증과 권한

인증 (Authentication) | 사용자가 주장하는 신원이 실제로 맞는지 확인하는 과정입니다.

인가 (Authorization) | 인증된 주체가 특정 데이터와 기능을 사용할 수 있는지 판단하는 과정입니다.

최소 권한 원칙 | 업무 수행에 필요한 최소한의 권한만 부여하는 보안 원칙입니다.

역할 기반 접근 제어 (RBAC) | 사용자에게 직접 권한을 부여하지 않고 직무 역할에 권한을 묶어 부여하는 방식입니다.

데이터 보호

암호화 (Encryption) | 평문 데이터를 키를 이용해 알아볼 수 없는 암호문으로 바꾸는 보호 방식입니다.

데이터 마스킹 | 데이터를 화면이나 출력에 표시할 때 민감한 부분을 가립니다.

데이터 비식별화 | 개인을 식별하기 어렵도록 식별자나 속성을 삭제·변환하는 처리입니다.

SQL Injection 방어 | 사용자 입력이 SQL 문법으로 실행되지 않도록 입력과 질의를 분리하는 보안 방법입니다.

감사 추적 (Audit) | 사용자의 로그인·조회·변경·권한 변경 활동을 기록하고 분석하는 기능입니다.

무결성과 품질

데이터 품질 | 데이터가 업무 목적에 맞게 정확하고 완전하며 일관되고 최신인지 평가하는 기준입니다.

참조 무결성 검증 | 외래키 값이 참조하는 테이블의 실제 키와 대응하는지 검사합니다.

도메인 무결성 검증 | 속성 값이 정해진 자료형·범위·형식·허용값을 만족하는지 검사합니다.

데이터 거버넌스 | 데이터의 소유자·관리자·표준·정책·품질 책임을 조직적으로 관리하는 체계입니다.

백업 유형

전체 백업 (Full Backup) | 선택한 데이터 전체를 한 번에 복사하는 백업입니다.

증분 백업 (Incremental) | 가장 최근에 수행한 전체 또는 증분 백업 이후 변경된 데이터만 저장합니다.

차등 백업 (Differential) | 가장 최근의 전체 백업 이후 변경된 데이터를 매번 누적해 저장합니다.

백업 검증 | 백업 파일이 실제로 복원 가능한지와 데이터가 손상되지 않았는지 확인하는 과정입니다.

복구 목표와 전략

RPO (Recovery Point Objective) | 장애가 발생했을 때 허용할 수 있는 데이터 손실의 최대 범위입니다.

RTO (Recovery Time Objective) | 장애 발생 후 서비스를 복구해야 하는 목표 시간입니다.

콜드 사이트 (Cold Site) | 재해 복구를 위한 장소와 기본 설비는 준비했지만 운영 시스템과 최신 데이터는 즉시 사용할 수 없는 대체 시설입니다.

웜 사이트 (Warm Site) | 일부 시스템과 데이터를 준비해 두고 재해 시 추가 복구 작업 후 서비스를 재개하는 시설입니다.

핫 사이트 (Hot Site) | 운영 환경과 최신 데이터를 상시 유지해 재해 발생 직후 빠르게 전환할 수 있는 대체 시설입니다.

고가용성 (HA) | 장애가 발생해도 서비스가 중단되지 않거나 빠르게 이어지도록 시스템을 이중화하는 구조입니다.

장애 대응 절차

탐지와 영향 분석 | 모니터링·알림·사용자 신고로 장애를 확인하고 영향 범위와 원인을 분류하는 단계입니다.

복구 순서 | 인프라·DB·메시지·애플리케이션처럼 의존 관계에 따라 서비스를 되살리는 절차입니다.

복구 후 검증 | 데이터 건수·합계·참조 관계·최근 거래·서비스 기능을 점검하는 단계입니다.

사후 분석 (Postmortem) | 장애 원인과 대응 과정, 재발 방지 조치를 기록하고 개선하는 활동입니다.

ETL과 매핑

추출 (Extract) | 원천 시스템에서 전환 대상 데이터를 읽어 중간 저장소나 변환 파이프라인으로 가져오는 단계입니다.
변환 (Transform) | 원천 데이터를 대상 시스템의 구조·자료형·코드·업무 규칙에 맞게 바꾸는 단계입니다.
적재 (Load) | 변환된 데이터를 대상 데이터베이스의 테이블과 관계에 맞게 저장하는 단계입니다.
데이터 매핑 | 원천 시스템의 테이블·컬럼·코드와 대상 시스템의 테이블·컬럼·규칙 사이의 대응 관계를 정의합니다.

품질 정제와 검증

데이터 프로파일링 | 원천 데이터의 분포·결측·중복·범위·형식·참조 상태를 조사하는 활동입니다.

데이터 정제 (Cleansing) | 오류·중복·누락·불일치 데이터를 표준 규칙에 맞게 수정하거나 분류하는 활동입니다.

대사 검증 (Reconciliation) | 전환 전후 데이터의 건수·합계·키·참조 관계와 업무 결과를 비교하는 검증입니다.

데이터 품질 차원 | 정확성·완전성·일관성·유일성·유효성·적시성 같은 기준으로 데이터 상태를 평가합니다.

시스템 전환 전략

빅뱅 전환 (Big Bang) | 특정 시점에 기존 시스템을 중단하고 새 시스템으로 한 번에 전환합니다.

단계적 전환 (Phased) | 업무·지역·기능 단위로 범위를 나누어 순서대로 전환합니다.

병행 운영 (Parallel) | 일정 기간 기존 시스템과 새 시스템을 함께 운영하며 결과를 비교합니다.

롤백 계획 | 전환 실패나 중대한 오류가 발생했을 때 이전 시스템과 데이터 상태로 되돌리는 절차입니다.

데이터베이스 구축

DB 설계 절차

요구사항→개념적→논리적→물리적→구현 설계 순서로
구체화합니다.

요구사항 분석 | 업무에서 필요한
데이터·처리·조회·갱신·보안·성능 요구를 수집하고 명세화합니다.

개념적 설계 | 업무에서 중요한 엔터티·속성·관계를 DBMS와
무관한 개념 모델로 표현합니다.

논리적 설계 | 개념 모델을 릴레이션·키·무결성 제약 등 논리적
데이터 구조로 변환합니다.

물리적 설계 | 논리적 구조를 실제 저장·접근 방식으로 구현하기
위한 인덱스·파티션·저장 공간을 결정합니다.

구현 | 설계한 스키마와 데이터베이스 객체를 DBMS에 실제로
생성하고 데이터를 적재합니다.

파일 접근 방식

순차 접근 | 파일의 시작부터 레코드를 저장 순서대로 읽으며 원하는 데이터를 찾는 방식입니다.

색인 순차 접근 | 색인에서 키와 위치를 찾은 뒤 실제 데이터 영역으로 이동하는 방식입니다.

해시 접근 | 검색 키를 해시 함수에 넣어 저장 주소를 계산하고 해당 위치에서 데이터를 찾는 방식입니다.

트랜잭션 상태

Active (활동) | 트랜잭션이 읽기·쓰기 명령을 실행 중인 상태입니다.

Partially Committed (부분 완료) | 마지막 명령은 끝났지만 커밋의 영구 반영이 아직 확정되지 않은 상태입니다.

Committed | 트랜잭션의 모든 작업이 성공해 결과가 영구적으로 반영된 상태입니다.

Failed (실패) | 오류나 장애로 트랜잭션을 더 이상 정상 실행할 수 없는 상태입니다.

Aborted (철회) | 실패한 트랜잭션의 변경을 롤백해 작업을 취소한 상태입니다.

격리 수준과 OLAP

Read Uncommitted | 다른 트랜잭션이 아직 커밋하지 않은 변경도 읽을 수 있는 격리 수준입니다.

Read Committed | 커밋된 데이터만 읽을 수 있는 격리 수준입니다.

Repeatable Read | 같은 트랜잭션 안에서 이미 읽은 행의 값을 반복 조회해도 유지하는 격리 수준입니다.

Serializable | 동시 실행 결과가 트랜잭션을 순서대로 실행한 것과 같도록 보장하는 가장 강한 격리 수준입니다.

MOLAP | 미리 구성한 다차원 큐브에 분석 데이터를 저장하는 OLAP 방식입니다.

ROLAP | 관계형 데이터베이스의 테이블과 SQL을 이용하는 OLAP 방식입니다.

HOLAP | 다차원 큐브와 관계형 데이터베이스를 함께 사용하는 OLAP 방식입니다.

분산 DB 투명성의 구분

위치 투명성 (Location Transparency) | 데이터가 저장된 물리적 위치를 몰라도 접근할 수 있습니다.

분할 투명성 (Fragmentation Transparency) | 하나의 논리 데이터를 여러 단편으로 나눈 사실을 숨깁니다.

복제 투명성 (Replication Transparency) | 여러 데이터 사본을 하나의 논리 데이터처럼 사용합니다.

병행 투명성 (Concurrency Transparency) | 여러 트랜잭션의 동시 실행에도 일관된 결과를 유지합니다.

장애 투명성 (Failure Transparency) | 일부 장애를 시스템이 처리해 데이터의 일관된 사용을 지원합니다.

이주 투명성 (Migration Transparency) | 데이터나 자원이 이동해도 사용 방법·이름을 유지합니다.

병행 제어 기법 비교

낙관적 검증 (Optimistic Validation) | 일단 작업하고 반영 전에 충돌을 검사합니다.

타임스탬프 순서 제어 (Timestamp Ordering) | 트랜잭션의 논리적 시간 순서를 기준으로 충돌 연산을 제어합니다.

MVCC (다중 버전 동시성 제어) | 데이터의 여러 버전을 유지해 읽기 일관성을 제공합니다.

큰 로킹 단위 (Coarse Granularity) | 테이블 등 넓은 범위를 적은 수의 잠금으로 제어합니다.

작은 로킹 단위 (Fine Granularity) | 행 등 좁은 범위를 개별 잠금으로 제어합니다.

회복 기법의 갱신 시점

즉시 갱신 (Immediate Update) | 트랜잭션 확정 전에 변경 페이지가 저장장치에 반영될 수 있습니다.

지연 갱신 (Deferred Update) | 확정 전에는 변경을 실제 DB에 반영하지 않는 고전적 회복 방식입니다.

그림자 페이징 (Shadow Paging) | 원본 페이지 테이블을 보존하고 변경 페이지를 별도로 관리합니다.

체크포인트 (Checkpoint) | 회복 작업의 시작 범위를 줄이기 위해 상태를 기록하는 지점입니다.

데이터베이스 구축

SQL 최적화의 핵심 용어

옵티마이저 (Optimizer) | SQL을 처리할 접근 경로·조인 순서·연산 방법을 선택합니다.

RBO (규칙 기반 최적화) | 미리 정해진 우선순위 규칙으로 실행 경로를 고릅니다.

CBO (비용 기반 최적화) | 통계와 예상 자원 비용으로 후보 계획을 비교합니다.

실행 계획 (Execution Plan) | SQL의 접근·조인·정렬 등 처리 경로를 나타냅니다.

힌트 (Hint) | 최적화 선택에 영향을 주도록 SQL에 제공하는 지시입니다.

인덱스 종류의 선택 기준

B-tree 인덱스 (균형 트리) | 키의 정렬 관계를 유지하는 균형 트리 계열의 검색 구조입니다.

해시 인덱스 (Hash Index) | 해시값을 이용해 키와 연결된 저장 위치에 접근합니다.

비트맵 인덱스 (Bitmap Index) | 값별 해당 행 여부를 비트 집합으로 표현합니다.

함수 기반 인덱스 (Function-based Index) | 열의 원래 값이 아니라 함수·표현식의 결과를 인덱싱합니다.

복합 인덱스 (Composite Index) | 둘 이상의 열을 조합한 키로 구성합니다.

인덱스 스캔 방식

Index Unique Scan (유일 검색) | 유일성이 보장된 키의 동등 조건으로 최대 한 행을 찾습니다.

Index Range Scan (범위 검색) | 시작 키를 찾은 뒤 조건에 맞는 범위의 인덱스를 읽습니다.

Index Full Scan (인덱스 전체 검색) | 인덱스 항목을 처음부터 끝까지 읽습니다.

Index Skip Scan (선두 열 생략 검색) | 복합 인덱스의 선두 열 조건 없이도 하위 범위를 활용하는 방식입니다.

데이터베이스 구축

API·플러그인·TDE·하이브리드

API 방식 (Application API) | 애플리케이션이 암호 모듈을 호출해 암복호화합니다.

플러그인 방식 (Plug-in) | DB 서버에 설치한 암호 모듈을 통해 처리합니다.

TDE (투명한 데이터 암호화) | DBMS가 저장 데이터의 암복호화를 내부적으로 수행합니다.

하이브리드 방식 (Hybrid) | 애플리케이션과 DB 계층의 방식을 조합합니다.

NoSQL의 네 가지 저장 모델

Key-Value (키-값) | 고유 키와 연결된 값의 형태로 저장합니다.

Document (문서) | JSON과 유사한 문서 단위로 자료를 저장합니다.

Wide-column (와이드 컬럼) | 행 키와 컬럼 패밀리를 중심으로 데이터를 구성합니다.

Graph (그래프) | 노드·관계와 그 속성으로 데이터를 표현합니다.

BASE의 세 가지 성질

Basically Available (기본적 가용성) | 일부 장애나 불일치가 있어도 가능한 서비스를 제공하는 지향입니다.

Soft State (유연한 상태) | 복제·전파 과정에서 상태가 일시적으로 달라질 수 있습니다.

Eventual Consistency (최종 일관성) | 추가 변경이 멈추면 복제본들이 최종적으로 같은 상태에 수렴합니다.

분류·군집·연관·순차 패턴

분류 (Classification) | 알려진 범주를 학습해 새 자료의 범주를 예측합니다.

군집 (Clustering) | 정답 라벨 없이 유사한 특성의 자료를 묶습니다.

연관 규칙 (Association Rule) | 함께 나타나는 항목 사이의 관계를 찾습니다.

순차 패턴 (Sequential Pattern) | 시간 또는 사건 순서를 포함한 발생 패턴을 찾습니다.

외래키 삭제·갱신 옵션 비교

CASCADE (연쇄 처리) | 부모 삭제를 참조 중인 자식 행의 삭제로 전파합니다.

SET NULL (참조값 비우기) | 자식 행은 남기고 해당 외래키 값을 NULL로 변경합니다.

RESTRICT·NO ACTION (제약 검사) | 참조 행 때문에 무결성이 깨지면 작업을 허용하지 않습니다.

ON UPDATE (키 값 갱신) | 부모 키 값 변경에 적용할 동작을 지정합니다.

데이터베이스 구축

CREATE DOMAIN과 CHECK 조건 읽기

CREATE DOMAIN (도메인 정의) | 이름이 붙은 자료형·제약의 묶음을 만들어 여러 열에서 재사용합니다.

CHECK·VALUE (값 검사) | CHECK 괄호 안의 조건이 입력 값에 적용됩니다.

NULL과 별도 조건 | CHECK만으로 NULL까지 자동 금지된다고 단정하지 않습니다.

데이터베이스 구축

후보키로 판별하는 3NF와 BCNF

후보키와 주요 속성 | AB와 AC가 최소한의 식별자이며 A·B·C는 모두 주요 속성입니다.

3NF를 만족하는 이유 | C→B에서 C는 슈퍼키가 아니지만 B가 주요 속성이므로 3NF 예외가 적용됩니다.

BCNF를 위반하는 이유 | C의 폐포는 BC까지만 확장되어 A를 결정하지 못합니다.

자료형과 형 변환

정수 연산 | 정수형 피연산자끼리 계산할 때 결과의 자료형과 나눗셈 규칙을 확인합니다.

실수 연산 | 피연산자에 실수가 포함되면 소수 부분을 가진 결과가 만들어질 수 있습니다.

자동 형 변환 | 작은 범위의 자료형이나 정수가 더 넓은 자료형으로 계산 과정에서 자동 변환되는 현상입니다.

명시적 형 변환 (Casting) | 개발자가 원하는 자료형으로 값을 직접 변환하는 표현입니다.

문자와 ASCII | 문자는 내부적으로 숫자 코드와 대응하므로 문자 사이의 차이를 계산할 수 있습니다.

연산자와 우선순위

산술 연산자 | 덧셈·뺄셈·곱셈·나눗셈·나머지로 값을 계산합니다.

대입 연산자 | 계산한 값을 변수에 저장하거나 복합 대입으로 계산과 저장을 함께 수행합니다.

증감 연산자 | ++와 --는 값을 1만큼 변경하며 전위·후위 위치에 따라 식에 사용되는 시점이 달라집니다.

비교 연산자 | 두 값의 크기나 동등 여부를 비교해 참·거짓 결과를 만듭니다.

논리 연산자 | 참·거짓 조건을 AND·OR·NOT으로 결합합니다.

비트 연산자 | 정수의 이진수 비트마다 AND·OR·XOR·반전 또는 시프트를 수행합니다.

삼항 연산자 | 조건에 따라 두 표현식 중 하나의 결과를 선택합니다.

실행 추적 절차

변수 추적표 | 행 번호별로 주요 변수의 값을 기록해 재할당과 누적 결과를 확인하는 풀이 방법입니다.

실행 순서 | 조건문·반복문·함수 호출·반환의 흐름을 실제 실행 순서로 나열하는 과정입니다.

출력 버퍼와 형식 | print·printf의 줄바꿈·구분자·자리 표시자에 따라 최종 문자열을 조립합니다.

조건문과 분기

if·else if·else | 조건을 위에서부터 검사해 처음 참인 블록 또는 else 블록을 실행합니다.

switch·case | 하나의 식을 case 값과 비교해 일치하는 구간을 실행하는 분기문입니다.

삼항 조건식 | 조건에 따라 두 표현식 중 하나를 선택해 값을 반환하는 간결한 조건문입니다.

반복문과 흐름 제어

for 문 | 반복 횟수나 인덱스의 시작·종료·증감을 한 곳에 표현하는 반복문입니다.

while | 조건이 참인 동안 본문을 반복하며 실행 전에 조건을 먼저 검사합니다.

do-while | 본문을 먼저 실행하고 마지막에 반복 조건을 검사합니다.

break | 가장 가까운 반복문 또는 switch를 즉시 종료합니다.

continue | 현재 반복의 남은 문장을 건너뛰고 다음 반복으로 넘어갑니다.

함수와 재귀

값에 의한 전달 | 호출 시 인자의 값이 매개변수에 복사되어 함수 내부 변경이 원본 변수에 직접 반영되지 않는 방식입니다.

C 주소 전달 | C 함수에 변수의 주소를 포인터 매개변수로 전달해 원본 저장 위치를 다룹니다.

객체 참조 전달 | Java·Python에서 객체를 가리키는 참조를 매개변수로 전달합니다.

재귀 함수 | 함수가 자기 자신을 호출해 입력을 줄여 가며 문제를 해결하는 함수입니다.

호출 스택 | 함수 호출마다 매개변수·지역 변수·복귀 위치를 저장하는 실행 영역입니다.

배열과 인덱스

1차원 배열 | 같은 자료형의 값을 일렬로 저장하고 인덱스로 접근하는 자료구조입니다.

배열 복사와 교환 | 배열 요소를 임시 변수에 저장하거나 언어의 복사 기능으로 다른 배열에 옮기는 작업입니다.

역순 접근 | 배열이나 문자열을 마지막 인덱스부터 처음까지 거꾸로 읽습니다.

대칭 접근 | 앞쪽 i와 뒤쪽 n-1-i를 한 쌍으로 비교하거나 교환합니다.

문자열 처리

C 문자열과 널 문자 | C에서 문자열은 문자 배열 끝에 ₩0이 들어 있는 형태로 표현됩니다.

문자열 슬라이싱 | 문자열이나 리스트의 시작·끝·간격을 지정해 일부를 추출하는 문법입니다.

문자열 연결 | 여러 문자열을 하나로 이어 붙이는 작업입니다.

문자열 분리 | 구분자를 기준으로 문자열을 여러 부분으로 나누는 작업입니다.

문자열 비교 | 문자열의 내용 또는 참조를 비교하는 연산입니다.

2차원 배열

행 우선 순회 | 각 행을 선택한 뒤 해당 행의 모든 열을 순서대로 처리하는 방식입니다.

열 우선 순회 | 각 열을 선택한 뒤 모든 행을 순서대로 처리하는 방식입니다.

행렬 전치 | 2차원 배열의 행과 열을 바꾸어 (i, j)를 (j, i)로 옮깁니다.

대각선 접근 | 정사각 행렬에서 주대각선 i=j와 부대각선 i+j=n-1을 찾습니다.

경계 접근 | 2차원 배열의 첫·마지막 행과 열에 있는 원소를 선택합니다.

포인터 핵심

주소 연산자 (&) | 변수의 메모리 주소를 구하는 연산자입니다.

역참조 연산자 (*) | 포인터가 가리키는 주소의 실제 값을 읽거나 수정하는 연산자입니다.

포인터 연산 | 포인터에 정수를 더하거나 빼 자료형 크기만큼 다음 요소 주소로 이동하는 연산입니다.

NULL 포인터 | 유효한 객체를 가리키지 않는 포인터 상태를 표현하는 값입니다.

포인터와 함수·배열

값에 의한 전달 | 함수 호출 시 변수의 값만 복사해 매개변수에 전달하는 방식입니다.

주소에 의한 전달 | 변수의 주소를 포인터로 전달해 함수가 원본 메모리에 접근하는 방식입니다.

배열 이름과 포인터 | 배열 이름은 대부분의 식에서 첫 번째 요소를 가리키는 주소처럼 사용됩니다.

문자 포인터와 문자 배열 | 문자 배열은 수정 가능한 저장 공간이고 문자열 리터럴을 가리키는 포인터는 수정하면 안 되는 경우가 많습니다.

구조체와 static

구조체 (struct) | 서로 다른 자료형의 멤버를 하나의 사용자 정의 자료형으로 묶습니다.

**멤버 접근 .·-> ** | 구조체 변수는 점 연산자, 구조체 포인터는 화살표 연산자로 멤버에 접근합니다.

static 지역 변수 | 함수 호출이 끝나도 값과 저장 공간이 유지되는 지역 변수입니다.

C의 static 전역 변수 | 파일 범위 변수에 static을 붙여 이름의 연결 범위를 해당 소스 파일 안으로 제한합니다.

Java의 static 메서드 | 객체가 아니라 클래스에 속하는 메서드입니다.

정렬 알고리즘

버블 정렬 (Bubble Sort) | 서로 이웃한 원소를 비교·교환해 큰 값 또는 작은 값을 반복해서 끝으로 이동합니다.

선택 정렬 (Selection Sort) | 정렬되지 않은 구간에서 최솟값 또는 최댓값을 찾아 앞의 위치와 교환합니다.

삽입 정렬 (Insertion Sort) | 앞쪽의 정렬된 구간에 현재 원소가 들어갈 위치를 찾아 삽입합니다.

정렬 방향과 조건 | 비교식의 부등호와 반복 범위가 오름차순·내림차순과 정렬 완료 시점을 결정합니다.

선형 자료구조

스택 (Stack) | 마지막에 넣은 값부터 꺼내는 LIFO 방식으로 호출 복귀나 실행 취소에 씁니다.

큐 (Queue) | 먼저 넣은 값부터 꺼내는 FIFO 방식으로 요청을 대기 순서대로 처리합니다.

연결 리스트 | 각 노드가 다음 노드 주소를 저장하며 연결을 따라 한 방향으로 이동합니다.

원형 연결 리스트 | 마지막 노드가 처음 노드를 가리켜 끝에서 시작으로 이어지는 순환 구조입니다.

이중 연결 리스트 | 각 노드가 이전·다음 주소를 저장하여 현재 노드에서 양쪽 방향으로 이동합니다.

트리와 탐색

트리 | 노드와 간선으로 부모·자식 관계를 나타내는 계층형 자료구조입니다.

이진 트리 | 각 노드가 최대 두 개의 자식 노드를 가진 트리입니다.

전위 순회 (Preorder) | 현재 노드→왼쪽 서브트리→오른쪽 서브트리 순서로 방문합니다.

중위 순회 (Inorder) | 왼쪽 서브트리→현재 노드→오른쪽 서브트리 순서로 방문합니다.

후위 순회 (Postorder) | 왼쪽 서브트리→오른쪽 서브트리→현재 노드 순서로 방문합니다.

선형 탐색 | 앞에서부터 원소를 하나씩 비교해 목표 값을 찾습니다.

이진 탐색 | 정렬된 배열에서 중간값과 비교하며 탐색 범위를 절반씩 줄입니다.

Python 컬렉션

list (리스트) | 순서와 중복값을 유지하며 생성 후 요소를 추가하거나 수정할 수 있는 시퀀스입니다.

tuple (튜플) | 순서와 중복값을 유지하지만 생성 후 요소 참조를 바꿀 수 없는 불변 시퀀스입니다.

set (집합) | 원소를 중복 저장하지 않으며 위치보다 원소 포함 여부와 집합 연산을 다룹니다.

dict (딕셔너리) | 유일한 키에 값을 연결하며 같은 키로 저장하면 기존 값은 새 값으로 바뀝니다.

인덱싱과 슬라이싱

인덱싱 | 하나의 위치 번호로 문자열·리스트·튜플의 요소를 가져오는 연산입니다.

슬라이싱 | start부터 end 직전까지 step 간격으로 여러 요소를 추출하는 문법입니다.

언패킹 | 컬렉션의 요소를 여러 변수에 순서대로 나누어 대입하는 문법입니다.

반복 표현식

lambda | 이름을 붙이지 않고 한 줄 표현식으로 정의하는 익명 함수입니다.

map | 반복 가능한 자료의 각 요소에 함수를 적용해 변환한 결과를 만듭니다.

filter | 반복 가능한 자료에서 조건 함수가 참인 요소만 남깁니다.

리스트 컴프리헨션 | 반복과 조건을 이용해 새 리스트를 간결하게 만드는 표현식입니다.

문자열 포맷팅 | 문자열 안에 변수·표현식·형식 지정 값을 삽입하는 방법입니다.

함수와 변수 범위

지역 변수 (Local) | 함수 호출 중에 만들어지고 해당 함수 범위에서만 접근할 수 있는 변수입니다.

전역 변수와 global | 모듈 또는 함수 밖에 선언되어 여러 함수에서 참조할 수 있는 변수와 이를 함수 안에서 재할당하는 선언입니다.

기본 인자와 반환 | 인자를 생략했을 때 사용할 기본값과 함수가 호출자에게 돌려주는 결과입니다.

클래스와 객체

class·self | class는 객체의 속성과 메서드 틀을 정의하고 self는 현재 인스턴스를 가리킵니다.

클래스 변수 | 클래스에 정의되어 여러 인스턴스가 함께 참조할 수 있는 변수입니다.

인스턴스 변수 | 각 객체가 독립적으로 가지는 속성입니다.

메서드 호출 | 객체가 메서드를 호출할 때 현재 객체가 self로 자동 전달되는 방식입니다.

복사와 참조

대입과 참조 공유 | 새 변수도 같은 객체를 가리키므로 한쪽에서 객체를 변경하면 다른 쪽에도 보입니다.

얕은 복사 (Shallow Copy) | 바깥 객체만 새로 만들고 중첩 객체는 공유하므로 내부 변경이 원본에도 반영됩니다.

깊은 복사 (Deep Copy) | 중첩 객체까지 재귀적으로 복제하여 복사본의 내부 변경이 원본과 분리됩니다.

구별: 중첩 리스트를 수정했을 때 얕은 복사본도 변하면, 안쪽 객체를 공유하고 있다는 뜻입니다.

클래스와 생성자

클래스와 객체 | 클래스는 속성과 메서드의 설계도이고 객체는 그 클래스로 만들어진 실제 인스턴스입니다.

생성자 (Constructor) | 객체가 생성될 때 상태를 초기화하는 특별한 메서드입니다.

this | 현재 객체의 필드·메서드나 현재 클래스의 다른 생성자를 참조하는 Java 키워드입니다.

super | 부모 클래스의 생성자·필드·메서드를 참조하는 Java 키워드입니다.

상속과 다형성

상속 (Inheritance) | 자식 클래스가 부모 클래스의 멤버와 규칙을 물려받아 확장하는 기능입니다.

오버라이딩 (Overriding) | 자식 클래스가 부모의 같은 메서드를 자신의 동작으로 재정의하는 방식입니다.

다형성 (Polymorphism) | 하나의 부모 타입이나 인터페이스로 여러 구체 객체를 다루는 성질입니다.

업캐스팅 (Upcasting) | 자식 클래스 객체를 부모 클래스 타입의 참조로 다룹니다.

다운캐스팅 (Downcasting) | 부모 타입 참조를 실제 자식 클래스 타입으로 변환합니다.

추상화와 계약

추상 클래스 (abstract class) | 직접 객체를 만들 수 없고 공통 구현과 추상 메서드를 함께 가질 수 있는 클래스입니다.

인터페이스 (interface) | 클래스가 제공해야 할 메서드 계약을 정의하는 타입입니다.

인터페이스 다형성 | 인터페이스 타입 참조로 여러 구현 객체를 동일한 방식으로 사용하는 구조입니다.

enum (열거형) | 정해진 이름의 상수 집합을 타입 안전하게 표현하는 문법입니다.

비교와 문자열

== 연산자 | 기본형에서는 값이 같은지, 참조형에서는 두 참조가 같은 객체를 가리키는지 비교합니다.

equals() | 객체가 정의한 논리적 내용이 같은지 비교하는 메서드입니다.

String 불변성 | Java String은 생성 후 내부 문자열 내용을 직접 바꿀 수 없는 불변 객체입니다.

static과 클래스 상태

static 변수 | 클래스에 하나만 만들어져 모든 인스턴스가 공유하는 변수입니다.

static 메서드 | 객체 생성 없이 클래스 이름으로 호출할 수 있는 메서드입니다.

final | 변수 재대입·메서드 재정의·클래스 상속을 제한하는 키워드입니다.

연산자와 분기 문법

증감 연산자 | ++, --로 변수 값을 1 늘리거나 줄입니다.

복합 대입 연산자 | +=, -=, *= 등으로 계산 결과를 같은 변수에 다시 저장합니다.

비트 논리 연산 | 정수의 이진 표현을 자리별로 AND·OR·XOR합니다.

비트 시프트 연산 | 정수의 비트 패턴을 왼쪽·오른쪽으로 이동합니다.

switch fall-through | case에 진입한 뒤 break가 없으면 아래 case의 문장도 이어서 실행됩니다.

배열 기본값과 길이 | Java 배열은 생성 시 자료형별 기본값으로 초기화되고 length로 크기를 확인합니다.

예외 흐름

try·catch | 예외가 발생할 수 있는 코드를 try에 두고 종류별 처리 코드를 catch에 작성합니다.

finally | 예외 발생 여부나 catch 실행 여부와 관계없이 마무리 작업을 수행하는 블록입니다.

예외 상속과 catch 순서 | 구체적인 예외는 넓은 상위 예외 타입에 포함될 수 있습니다.

throw | Java 코드에서 예외 객체를 직접 발생시키는 키워드입니다.

throws | Java 메서드 선언부에 발생 가능 예외를 명시해 호출자에게 처리를 맡기는 키워드입니다.

오버로딩과 오버라이딩

오버로딩 (Overloading) | 이름은 같지만 매개변수 타입·개수가 달라 전달한 인자에 맞는 메서드를 고릅니다.

오버라이딩 (Overriding) | 상속받은 메서드를 같은 시그니처로 다시 정의하여 하위 클래스의 동작을 바꿉니다.

정적 바인딩 (Static Binding) | 선언된 참조 타입과 인자 타입을 보고 컴파일 시 호출할 시그니처를 선택합니다.

동적 바인딩 (Dynamic Binding) | 실행 중 실제 객체의 타입을 보고 재정의된 인스턴스 메서드 구현을 선택합니다.

구별: 오버로딩은 인자 타입·개수, 오버라이딩은 실제 객체 타입이 핵심 단서입니다.

실행 순서와 재귀

초기화·생성 순서 | 클래스 로딩·필드 초기화·생성자 호출이 정해진 순서로 실행됩니다.

재귀 호출 추적 | 메서드가 자기 자신을 호출하는 동안 각 호출의 매개변수와 반환값을 스택에 쌓습니다.

출력 순서 검산 | println·print·문자열 연결·메서드 호출의 평가 순서를 실제 화면 순서로 조립합니다.

동적 메모리

malloc | C에서 지정한 바이트 수의 동적 메모리를 할당합니다.

calloc | C에서 요소 개수와 크기를 받아 동적 메모리를 할당하고 모든 비트를 0으로 초기화합니다.

realloc | 기존 동적 영역의 크기를 바꾸고 필요하면 새 영역으로 이동시킨 뒤 주소를 반환합니다.

free와 메모리 오류 | free는 동적 할당 영역을 반환하며 이중 해제·해제 후 사용·누수는 오류 원인이 됩니다.

동적 2차원 배열 | 행 포인터 배열과 각 행의 데이터 영역을 따로 할당하거나 연속 공간을 인덱스로 나누어 사용합니다.

함수 포인터

선언·대입·호출 | 함수 포인터는 함수의 반환형과 매개변수형을 포함해 선언하고 같은 형식의 함수 주소를 대입합니다.

콜백과 배열 | 함수 포인터를 매개변수로 전달하면 호출하는 쪽이 처리 함수를 선택하는 콜백 구조를 만들 수 있습니다.

구조체 함수 포인터 | 구조체 멤버에 함수 포인터를 넣어 객체처럼 동작하는 인터페이스를 구성할 수 있습니다.

람다와 함수형 인터페이스

Functional Interface | 추상 메서드를 하나만 가지며 람다식으로 구현할 수 있는 인터페이스입니다.

Lambda Expression | 이름 없는 함수 구현을 화살표 문법으로 표현하는 방식입니다.

예외와 람다 | 람다에서 발생하는 예외는 대상 함수형 인터페이스의 throws 선언과 catch 위치에 영향을 받습니다.

static·오버라이딩·예외

Static Method Hiding | 하위 클래스에 같은 이름의 static 메서드가 있어도 인스턴스 메서드처럼 재정의되지 않습니다.

Instance Method Overriding | 상위 클래스의 일반 메서드를 하위 클래스가 같은 시그니처로 재정의하면 실제 객체의 메서드가 실행됩니다.

try | Java에서 예외가 발생할 수 있는 코드를 감싸는 블록입니다.

catch | 발생한 예외 중 선언한 타입에 맞는 예외를 처리하는 블록입니다.

finally | 예외 처리 뒤 정리 작업을 수행하는 블록입니다.

throws | 메서드 선언에서 호출자에게 전달할 수 있는 예외를 명시합니다.

프로그래밍 언어 활용

Java List·Set·Map 구현

ArrayList (배열 리스트) | 인덱스 순서와 중복 원소를 유지하는 동적 배열 기반 List입니다.

LinkedList (연결 리스트) | 노드의 연결로 순서를 유지하며 중복을 허용하는 List입니다.

HashSet (해시 집합) | 중복을 허용하지 않으며 원소의 순회 순서를 보장하지 않습니다.

HashMap (해시 맵) | 고유한 키에 값을 연결하며 순회 순서를 보장하지 않습니다.

프로그래밍 언어 활용

명령형·객체지향·함수형·선언형

절차적 프로그래밍 (Procedural) | 실행할 명령과 절차의 순서로 문제를 풉니다.

객체지향 프로그래밍 (Object-oriented) | 상태와 동작을 가진 객체들의 협력으로 문제를 모델링합니다.

함수형 프로그래밍 (Functional) | 함수의 적용과 조합으로 계산을 표현합니다.

논리형 프로그래밍 (Logic) | 사실과 규칙을 기술하고 추론으로 답을 찾습니다.

선언형 표현 (Declarative) | 세부 실행 순서보다 원하는 결과나 관계를 기술합니다.

마크업 언어 (Markup) | 문서의 구조·의미·데이터 형식을 태그 등으로 표현합니다.

개발 도구의 역할과 IoC

라이브러리 (Library) | 프로그램에서 필요한 기능을 호출해 사용하는 코드 집합입니다.

프레임워크 (Framework) | 애플리케이션의 구조와 공통 실행 흐름을 제공하는 기반입니다.

IoC (제어의 역전) | 객체 생성이나 호출 흐름의 제어를 외부 구성에 맡기는 원리입니다.

런타임 (Runtime) | 프로그램이 실행되는 데 필요한 실행 환경입니다.

C 문자 판별과 ASCII 범위 변환

isupper·islower (영문 판별) | 대문자인지 소문자인지 검사하며 분기마다 다른 기준 문자를 사용합니다.

isdigit (숫자 문자 판별) | '7' 같은 숫자 문자와 정수 7은 다른 값입니다.

나머지와 순환 범위 | 영문 전체를 순환시킬 때는 보통 26개 문자의 범위를 사용합니다.

C 출력 서식과 16진수 해석

%u·%x·%X (부호 없는 정수) | unsigned int 값의 10진수·소문자 16진수·대문자 16진수를 표시합니다.

%c·%s (문자와 문자열) | %c는 문자 코드 하나를, %s는 문자 배열을 따라 널 문자 직전까지 출력합니다.

서식 폭·주소 구분 | %04x는 최소 네 자리 폭을 0으로 채워 출력하는 예입니다.

프로그래밍 언어 활용

C 구조체 복사와 포인터 멤버의 공유

일반 멤버의 복사 | b.value를 바꿔도 별도 구조체인 a.value가 자동으로 바뀌지는 않습니다.

포인터 대상의 공유 | *b.link를 바꾸면 처음에는 a.link로 읽는 값도 달라집니다.

포인터 재대입 | b.link=&b.value처럼 주소를 바꾸면 해당 포인터 멤버의 연결만 달라집니다.

Java 제네릭과 오버로딩의 선택 시점

제한 없는 T | class Box<T>의 T에는 별도 상한이 없으므로 Object 기준으로 컴파일합니다.

호출 지점의 선언 타입 | main에서 Box<String>.value를 읽으면 그 식의 타입은 String입니다.

상한과 재정의 구분 | T extends Number의 상한은 Number이며 T를 항상 Object로만 처리하는 것은 아닙니다.

프로그래밍 언어 활용

Java enum의 순서·ordinal·values

ordinal() (선언 위치) | LOW는 0, MID는 1, HIGH는 2라는 순서 번호를 갖습니다.

values() (상수 배열) | 모든 상수를 선언 순서대로 담은 배열을 반환합니다.

이름·별도 필드 | name()은 선언한 상수 이름이고 개발자가 만든 code 필드는 별도 값입니다.

Java static 문맥의 컴파일 오류 판별

객체 없는 인스턴스 접근 | static main에서 count를 바로 읽으면 대상 객체가 없어 컴파일 오류가 납니다.

객체를 지정한 접근 | Sample s=new Sample(); 뒤의 s.count는 특정 객체의 필드를 읽습니다.

static 멤버와 판단 순서 | Sample.total은 객체를 만들지 않고 클래스 멤버를 읽는 표현입니다.

운영체제 역할

프로세스 관리 | 프로그램의 실행 단위인 프로세스를 생성·스케줄링·동기화·종료하는 기능입니다.

기억장치 관리 | 주기억장치와 가상 메모리를 관리해 프로세스에 필요한 공간을 할당·회수하는 기능입니다.

파일 관리 | 파일과 디렉터리의 생성·삭제·저장·접근 권한을 관리하는 운영체제 기능입니다.

입출력 관리 | 장치와 프로그램 사이의 데이터 송수신을 제어하는 운영체제 기능입니다.

커널과 쉘 | 커널은 운영체제의 핵심 자원 관리 부분이고 쉘은 사용자의 명령을 해석해 실행하는 인터페이스입니다.

운영체제 종류

Windows | Microsoft가 개발한 개인용·서버용 운영체제로 GUI와 다양한 응용 프로그램 지원이 강점입니다.

Unix | 다중 사용자·다중 작업을 지원하는 운영체제 계열입니다.

Linux | Unix와 유사한 기능을 제공하는 오픈 소스 커널 기반 운영체제 계열입니다.

Android | Linux 커널을 기반으로 모바일 장치의 앱과 자원을 관리하는 운영체제입니다.

iOS | Apple 모바일 기기에서 앱 실행과 하드웨어 자원을 관리하는 운영체제입니다.

Unix/Linux 명령

경로 확인·이동 명령 | pwd는 현재 디렉터리를 출력하고 cd는 작업 디렉터리를 바꿉니다.

목록 명령 | ls는 현재 또는 지정한 디렉터리의 파일·디렉터리 목록을 보여줍니다.

파일 조작 명령 | cp는 복사, mv는 이동·이름 변경, rm은 삭제, mkdir은 디렉터리 생성을 수행합니다.

권한과 프로세스 명령 | chmod는 파일 권한, chown은 소유자, ps·kill은 프로세스 조회·종료에 사용됩니다.

프로세스 상태

프로세스 상태 | 생성·준비·실행·대기·종료 상태로 프로세스의 실행 단계를 표현합니다.

PCB (Process Control Block) | 프로세스의 상태·프로그램 카운터·레지스터·스케줄링 정보 등을 저장하는 제어 블록입니다.

문맥 교환 (Context Switch) | 실행 중인 프로세스의 상태를 저장하고 다른 프로세스의 상태를 복원하는 과정입니다.

스케줄링 알고리즘

FCFS (First Come First Served) | 먼저 도착한 프로세스부터 CPU를 배정하며 실행이 끝날 때까지 선점하지 않습니다.

SJF (최단 작업 우선) | 준비된 작업 중 실행 시간이 가장 짧은 것을 골라 완료될 때까지 계속 처리합니다.

SRTF (최소 잔여 시간 우선) | 남은 실행 시간이 가장 짧은 작업을 고르고 더 짧은 새 작업이 오면 선점합니다.

우선순위 스케줄링 (Priority) | 지정한 우선순위가 높은 작업을 고르며 방식에 따라 실행 중인 작업을 선점합니다.

HRN (최고 응답률 우선) | (대기+실행)/실행의 응답률로 대기 정도를 반영하여 가장 큰 값을 고릅니다.

Round Robin (RR) | 같은 시간 할당량만큼 실행하고 미완료 작업은 큐 뒤로 보내 순환해서 처리합니다.

구별: SJF와 SRTF는 모두 짧은 작업을 선호하지만, SRTF만 새 작업 도착 시 현재 작업을 선점합니다.

스케줄링 계산

대기시간 | 프로세스가 준비 큐에서 CPU를 기다린 총 시간입니다.

반환시간 (Turnaround Time) | 프로세스가 도착한 시점부터 실행을 완전히 마칠 때까지 걸린 시간입니다.

응답시간 (Response Time) | 프로세스가 도착한 시점부터 처음 CPU를 할당받기까지 걸린 시간입니다.

간트 차트 | 시간축에 프로세스의 실행 순서와 구간을 막대 형태로 나타낸 표입니다.

교착 상태

상호 배제 (Mutual Exclusion) | 한 번에 하나의 프로세스만 사용할 수 있는 자원이 존재하는 조건입니다.
점유와 대기 (Hold and Wait) | 일부 자원을 점유한 채 다른 자원의 할당을 기다리는 조건입니다.
비선점 (No Preemption) | 할당된 자원을 다른 주체가 강제로 회수할 수 없는 조건입니다.
환형 대기 | 프로세스들이 원형으로 서로의 자원을 기다리는 조건입니다.

교착 해결

교착 상태 예방 | 네 가지 필요 조건 중 하나 이상이 성립하지 않도록 자원 요청 규칙을 설계합니다.

은행원 알고리즘 | 자원을 할당하기 전에 남은 자원으로 모든 프로세스가 완료 가능한 안전 상태인지 검사하는 회피 알고리즘입니다.

탐지와 복구 | 대기 그래프·자원 할당 그래프로 교착을 발견하고 프로세스 종료·자원 회수·롤백으로 복구합니다.

메모리·파일 관리

페이징·페이지 테이블 | 프로세스의 논리 메모리를 고정 크기 페이지로, 물리 메모리를 프레임으로 나누어 매핑합니다.

페이지 교체 알고리즘 | 페이지 부재가 발생해 빈 프레임이 없을 때 내보낼 페이지를 선택하는 방식입니다.

파일 관리 | 파일의 이름·속성·저장 위치·접근 권한과 디렉터리 구조를 관리하는 기능입니다.

OSI와 TCP/IP

물리·링크·네트워크·전송·세션·표현·응용 순서로 아래에서 위로 올라갑니다.

물리 계층 (1계층) | 비트를 전기·광·무선 신호로 바꾸어 전송 매체로 전달합니다.

데이터링크 계층 (2계층) | 같은 링크의 노드 사이에서 프레임을 전달하고 MAC 주소와 오류 검출을 다룹니다.

네트워크 계층 | 서로 다른 네트워크 사이에서 IP 주소를 이용해 패킷의 경로를 선택하고 전달합니다.

전송 계층 | 송신 프로세스와 수신 프로세스 사이의 종단 간 전송, 포트, 흐름·오류·혼잡 제어를 담당합니다.

세션 계층 (5계층) | 응용 프로그램 사이의 대화 연결을 설정·유지·종료하고 대화 흐름을 관리합니다.

표현 계층 (6계층) | 응용 데이터의 표현 형식을 맞추고 인코딩·압축·암호화 같은 변환을 다룹니다.

응용 계층 (7계층) | 사용자 응용 프로그램에 네트워크 서비스를 제공하는 최상위 계층입니다.

토폴로지

버스형 (Bus) | 하나의 공통 전송 회선에 여러 노드를 연결하는 토폴로지입니다.

스타형 (Star) | 중앙 허브·스위치에 각 노드를 연결하는 토폴로지입니다.

링형 토폴로지 | 노드들이 고리 모양으로 연결되는 네트워크 구조입니다.

망형 토폴로지 | 노드들을 여러 경로로 서로 연결하는 네트워크 구조입니다.

트리형 토폴로지 | 상위 노드에서 하위 노드로 계층적으로 분기하는 네트워크 구조입니다.

네트워크 장비

허브 (Hub) | 들어온 신호를 다른 포트로 전달하며 MAC 주소로 목적지 포트를 고르지 않습니다.

리피터 (Repeater) | 약해진 신호를 재생하여 전달 거리를 늘리며 주소를 보고 목적지를 고르지 않습니다.

브리지 (Bridge) | MAC 주소를 학습하여 LAN 구간 사이에서 프레임을 전달하거나 걸러 냅니다.

스위치 (Switch) | MAC 주소로 목적지 포트를 골라 프레임을 전달하는 다중 포트 연결 장비입니다.

라우터 (Router) | 목적지 IP 주소와 라우팅 정보를 보고 다른 네트워크로 패킷을 전달합니다.

게이트웨이 (Gateway) | 다른 통신 형식이나 프로토콜을 변환하여 직접 통신하기 어려운 둘을 연결합니다.

IPv4·IPv6와 서브네팅

IPv4	32비트 주소를 네 개의 8비트 옥텟으로 표현하는 주소 체계입니다.
IPv6	128비트 주소를 사용해 IPv4의 주소 고갈을 해결하고 자동 설정·확장 헤더를 지원하는 주소 체계입니다.
서브네팅 (Subnetting)	하나의 네트워크를 호스트 비트 일부를 빌려 여러 작은 네트워크로 분할하는 작업입니다.
네트워크 주소	서브넷 전체를 식별하는 기준 주소입니다.
브로드캐스트 주소	해당 서브넷의 모든 호스트에 보내는 데 사용하는 주소입니다.

주소 서비스

DNS (Domain Name System) | 도메인에 대응하는 IP 주소를 조회하여 접속할 서버의 네트워크 위치를 찾습니다.

DHCP | IP 주소·마스크·게이트웨이 등을 임대하여 호스트의 네트워크 설정을 자동화합니다.

NAT (주소 변환) | 경계에서 패킷의 IP 주소를 변환하여 사설망과 외부망 사이의 통신을 연결합니다.

PAT (포트 주소 변환) | IP와 포트를 함께 변환하여 여러 내부 연결이 하나의 공인 IP를 공유합니다.

구별: PAT는 NAT의 한 형태입니다. 여러 내부 기기가 하나의 공인 IP를 공유하는 예라면 포트 변환 여부도 확인하세요.

라우팅 프로토콜

RIP | 같은 AS 내부에서 거치는 라우터의 홉 수를 비교하여 가장 적은 경로를 고릅니다.

OSPF | 같은 AS 내부의 링크 상태를 공유하여 비용의 합이 가장 작은 경로를 구합니다.

BGP | 다른 AS 사이에서 경로 속성을 교환하며 조직의 정책을 반영해 경로를 고릅니다.

구별: RIP의 홉 수가 적은 경로와 OSPF의 비용이 낮은 경로는 같은 경로가 아닐 수 있습니다. BGP는 두 방식의 단순 최단 경로 알고리즘이 아닙니다.

TCP와 UDP

TCP | 연결을 맺고 순서 번호와 재전송으로 데이터가 빠짐없이 순서대로 도착하게 합니다.

UDP | 연결 없이 보내 지연을 줄이지만 유실 복구나 순서 보장은 자체 제공하지 않습니다.

포트 (Port) | 한 호스트 안에서 통신할 서비스나 프로세스를 구분하기 위해 사용하는 숫자입니다.

소켓 (Socket) | IP와 포트 등으로 구성된 통신 끝점으로 데이터를 주고받을 대상을 나타냅니다.

응용 프로토콜

HTTP | 웹 클라이언트와 서버가 요청·응답으로 자원을 주고받는 응용 계층 프로토콜입니다.

HTTPS | HTTP 통신을 TLS로 보호해 전송 구간을 암호화하는 방식입니다.

DNS | 도메인 이름에 대응하는 IP 주소 등을 조회하는 이름 해석 서비스입니다.

DHCP | 호스트에 IP 주소와 기본 게이트웨이·DNS 서버 등의 네트워크 설정을 자동으로 제공합니다.

FTP | 클라이언트와 서버 사이에 파일을 전송하는 응용 계층 프로토콜입니다.

SMTP | 전자우편을 보내거나 메일 서버 사이에 전달하는 프로토콜입니다.

POP3 | 메일 서버에서 전자우편을 받아오는 수신 프로토콜입니다.

IMAP | 메일을 서버에 두고 폴더·읽음 상태 등을 여러 장치와 동기화하는 수신 프로토콜입니다.

RTP | 실시간 음성·영상 데이터를 전달하는 프로토콜입니다.

RTCP | RTP 전송의 품질과 제어 정보를 교환하는 프로토콜입니다.

통신 제어

오류 제어 | 전송 중 손상·손실·중복된 데이터를 검출하고 재전송이나 복구로 신뢰성을 높이는 기능입니다.

흐름 제어 | 송신 속도를 수신자가 처리할 수 있는 속도에 맞추는 기능입니다.

혼잡 제어 | 네트워크에 패킷이 과도하게 몰려 지연·손실이 커지는 것을 조절하는 기능입니다.

HDLC | 동기식 링크에서 프레임 단위로 데이터를 전달하는 데이터링크 계층 프로토콜입니다.

PPP | 두 지점 사이의 링크에서 네트워크 계층 패킷을 캡슐화해 전달하는 프로토콜입니다.

보안의 기본 목표

기밀성 (Confidentiality) | 허가되지 않은 주체가 정보를 읽지 못하도록 보호하는 성질입니다.

무결성 (Integrity) | 허가되지 않은 변경이나 파괴 없이 데이터가 정확하고 일관된 상태를 유지하는 성질입니다.

가용성 (Availability) | 허가된 사용자가 필요할 때 시스템과 데이터에 접근할 수 있는 성질입니다.

부인방지 (Non-repudiation) | 송·수신자가 특정 행위나 메시지 전송 사실을 나중에 부인하지 못하게 하는 성질입니다.

AAA와 공격 분류

인증 (Authentication) | 사용자가 주장하는 신원이 실제 본인인지 확인합니다.

인가 (Authorization) | 인증된 사용자가 특정 자원에 접근하거나 작업할 권한이 있는지 판단합니다.

책임 추적 (Accountability) | 누가 언제 어떤 자원에 접근하고 작업했는지 기록·추적합니다.

DoS (서비스 거부) | 서버나 네트워크의 자원을 고갈시켜 정상 사용자의 접속을 막는 공격입니다.

DDoS (분산 서비스 거부) | 여러 분산된 공격원이 동시에 대상의 가용성을 떨어뜨리는 공격입니다.

스푸핑 (Spoofing) | 다른 주체의 IP·MAC·이메일 주소나 신원을 사칭하는 공격입니다.

스니핑 (Sniffing) | 네트워크를 오가는 패킷을 가로채거나 관찰해 정보를 수집하는 행위입니다.

피싱 (Phishing) | 가짜 메시지나 사이트로 사용자를 속여 비밀번호·금융 정보를 입력하게 하는 공격입니다.

파밍 (Pharming) | 주소 입력이나 이름 해석 과정을 조작해 정상 주소를 입력해도 가짜 사이트로 보내는 공격입니다.

네트워크·웹 공격

플러딩 (Flooding) | 대량의 패킷이나 요청을 보내 네트워크·서버 자원을 소진시키는 공격 방식입니다.

스머핑 (Smurfing) | 피해자 IP를 출발지로 위조한 ICMP 요청을 브로드캐스트 주소에 보내 응답을 피해자에게 집중시키는 공격입니다.

랜드 공격 (LAND) | 패킷의 출발지와 목적지 IP·포트를 피해자 자신의 값으로 위조해 처리에 부담을 주는 공격입니다.

XSS (Cross-Site Scripting) | 공격자가 입력한 스크립트가 다른 사용자의 브라우저에서 실행되도록 하는 웹 공격입니다.

SQL Injection | 입력값에 SQL 구문을 삽입해 인증 우회·정보 조회·데이터 변경을 시도하는 공격입니다.

무차별 대입 공격 (Brute Force) | 가능한 비밀번호 조합을 차례로 시도해 인증을 통과하려는 공격입니다.

사전 공격 (Dictionary Attack) | 자주 쓰는 단어나 유출된 비밀번호 목록을 우선 시도하는 공격입니다.

암호 방식

대칭키 암호 | 같은 비밀키로 암호화·복호화하여 빠르게 처리하지만 안전한 키 공유가 필요합니다.

공개키 암호 | 공개키와 개인키를 한 쌍으로 사용하여 공개된 키와 비밀키의 역할을 나눕니다.

하이브리드 암호 | 세션키는 공개키 방식으로 전달하고 본문은 빠른 대칭키 방식으로 암호화합니다.

해시·서명·인증서

해시 함수 | 입력 데이터를 고정 길이의 값으로 변환하며 같은 입력은 같은 결과, 작은 변경은 다른 결과를 만드는 함수입니다.

전자서명 | 송신자가 개인키로 메시지의 해시 등에 서명하고 수신자가 공개키로 검증하는 기술입니다.

공개키 인증서·PKI | 인증기관이 공개키와 소유자 정보를 전자적으로 서명해 공개키의 진위를 보증하는 체계입니다.

Salt | 비밀번호 해시에 사용자별 임의 값을 덧붙여 같은 비밀번호도 서로 다른 해시가 되게 하는 값입니다.

보안 프로토콜

SSL/TLS | 웹 등 응용 프로토콜의 통신 구간을 암호화하고 서버 인증과 무결성을 제공하는 보안 프로토콜입니다.

IPSec | IP 계층에서 패킷을 인증·암호화해 네트워크 구간을 보호하는 프로토콜 모음입니다.

SSH | 원격 호스트 접속과 명령 실행, 파일 전송을 암호화하는 보안 프로토콜입니다.

WPA | WEP의 취약점을 보완하기 위해 도입된 무선 LAN 보안 규격입니다.

WPA2 | AES 기반 CCMP를 채택한 무선 LAN 보안 규격입니다.

WPA3 | 향상된 인증·암호화 보호를 제공하는 무선 LAN 보안 규격입니다.

접근 통제 모델

DAC (임의 접근 통제) | 자원의 소유자가 허용 대상을 정하고 자신의 판단에 따라 접근 권한을 부여합니다.

MAC (강제 접근 통제) | 중앙의 보안 등급과 정책이 접근을 결정하므로 소유자가 임의로 허용할 수 없습니다.

RBAC (역할 기반 접근 통제) | 직무·역할에 권한을 묶어 부여하고 사용자가 맡은 역할을 기준으로 접근을 정합니다.

ABAC (속성 기반 접근 통제) | 사용자·자원·시간·위치 등 속성을 정책 규칙과 비교하여 현재 접근을 결정합니다.

방화벽·탐지·차단

방화벽 (Firewall) | 허용·차단 정책에 따라 네트워크 트래픽을 검사하고 내부 자원을 보호하는 시스템입니다.

IDS (Intrusion Detection System) | 침입이나 이상 행위를 탐지해 관리자에게 경보와 로그를 제공하는 시스템입니다.

IPS (Intrusion Prevention System) | 침입을 탐지하고 정책에 따라 공격 트래픽을 자동으로 차단하는 시스템입니다.

WAF (Web Application Firewall) | HTTP·HTTPS 요청을 분석해 웹 애플리케이션 공격을 필터링하는 방화벽입니다.

안전한 연결

VPN (Virtual Private Network) | 공용 네트워크 위에 인증·암호화된 터널을 만들어 사설망처럼 통신하는 기술입니다.

NAC (Network Access Control) | 네트워크에 접속하려는 사용자와 장치의 신원·보안 상태를 검사하는 기술입니다.

제로 트러스트 | 내부망이라는 이유로 신뢰하지 않고 모든 요청을 지속적으로 검증하는 보안 모델입니다.

정보시스템 구축 관리

서버와 저장장치

RAID 0 (Striping) | 데이터를 여러 디스크에 분산 기록해 성능과 용량을 높이는 구성입니다.

RAID 1 (Mirroring) | 같은 데이터를 두 디스크에 복제해 한 디스크 장애에도 서비스를 유지하는 구성입니다.

RAID 5 | 데이터와 패리티를 여러 디스크에 분산해 저장하는 구성입니다.

RAID 6 | 데이터와 두 종류의 패리티를 여러 디스크에 분산해 저장하는 구성입니다.

RAID 10 | 미러링한 디스크 쌍을 스트라이핑해 성능과 장애 내성을 함께 얻는 구성입니다.

스토리지 계층 | 자주 쓰는 데이터는 빠른 저장장치에, 대용량·장기 보관 데이터는 비용 효율적인 저장장치에 배치하는 구조입니다.

운영 관측과 복구

모니터링 | CPU·메모리·디스크·네트워크·응답 시간·오류율 같은 지표를 지속적으로 관찰하는 활동입니다.

로그 관리 | 시스템·애플리케이션·보안 이벤트를 시간·주체·내용과 함께 기록하고 분석하는 활동입니다.

백업·복구 | 장애나 삭제에 대비해 데이터 복사본을 만들고 정해진 시점·시간 안에 복원하는 절차입니다.

IT 서비스 관리

SLA (Service Level Agreement) | 서비스 제공자와 고객이 가용성·응답 시간·지원 범위·측정 방법을 합의한 계약입니다.

장애 관리 (Incident Management) | 중단된 서비스를 가능한 빨리 정상 상태로 복구하는 활동입니다.

문제 관리 (Problem Management) | 반복 장애의 근본 원인을 찾아 재발을 막는 활동입니다.

변경 관리 (Change Management) | 시스템 변경의 영향과 위험을 검토·승인하고 적용을 통제합니다.

구성 관리 (Configuration Management) | 서버·소프트웨어·네트워크 등 구성 항목의 기준 상태와 변경 이력을 관리합니다.

가상화

하이퍼바이저 | 물리 하드웨어의 자원을 가상 머신에 배분하고 실행을 격리하는 가상화 소프트웨어입니다.

전가상화 (Full Virtualization) | 게스트 운영체제를 수정하지 않고 가상 하드웨어 위에서 실행하는 방식입니다.

반가상화 (Paravirtualization) | 게스트 운영체제가 가상화 환경을 인식하고 하이퍼바이저와 협력하는 방식입니다.

가상 머신 (VM) | CPU·메모리·디스크·네트워크를 가상 장치로 제공해 독립 운영체제를 실행하는 환경입니다.

클라우드 서비스

IaaS | 서버·스토리지·네트워크를 제공하며 사용자가 운영체제와 애플리케이션을 관리합니다.

PaaS | 개발·실행 플랫폼을 제공하여 사용자가 애플리케이션 개발과 배포에 집중하게 합니다.

SaaS | 완성된 애플리케이션을 제공하여 사용자는 설치·운영 관리 없이 서비스로 이용합니다.

공개 클라우드 (Public) | 외부 사업자가 여러 고객에게 공유 인프라 기반의 자원을 제공하는 모델입니다.

사설 클라우드 (Private) | 한 조직만을 위해 전용으로 운영하는 클라우드 환경입니다.

하이브리드 클라우드 (Hybrid) | 공개·사설 클라우드를 연결해 업무에 따라 함께 사용하는 모델입니다.

FaaS·서버리스 | 이벤트나 요청에 따라 함수를 실행하며 서버와 실행 자원 관리는 제공자가 맡습니다.

구별: 공개·사설·하이브리드 클라우드는 누가 인프라를 사용하고 어디에 배치하는지의 축이므로 IaaS/PaaS/SaaS와 함께 조합될 수 있습니다.

컨테이너와 운영

컨테이너 | 애플리케이션과 라이브러리·설정·실행 파일을 격리된 프로세스로 묶어 실행하는 기술입니다.

컨테이너 이미지 | 컨테이너를 만들기 위한 불변 패키지로 애플리케이션과 실행 환경의 계층을 포함합니다.

오케스트레이션 | 여러 컨테이너의 배포·서비스 발견·로드 밸런싱·자동 확장·장애 복구를 관리하는 기능입니다.

웹 서비스 방식

SOAP | XML 기반 메시지와 표준 확장 규격을 이용해 서비스를 호출하는 웹 서비스 방식입니다.

WSDL | 웹 서비스의 연산·메시지 형식·접속 위치 등을 기술하는 문서 형식입니다.

UDDI | 웹 서비스를 등록하고 검색할 수 있도록 하는 디렉터리 표준입니다.

REST | 자원을 URI로 식별하고 HTTP 메서드 등으로 조작하는 웹 아키텍처 스타일입니다.

JSON | 객체·배열·문자열·숫자 등으로 데이터를 표현하는 텍스트 형식입니다.

메시지와 비동기 통합

메시지 큐 (MQ) | 송신자가 메시지를 큐에 넣고 수신자가 나중에 꺼내 처리하도록 중간 저장하는 통합 방식입니다.

발행·구독 (Pub/Sub) | 하나의 이벤트를 발행하면 해당 주제를 구독한 여러 소비자에게 전달하는 메시징 방식입니다.

멱등성과 재처리 | 같은 메시지를 여러 번 처리해도 최종 결과가 한 번 처리한 것과 같도록 만드는 성질입니다.

통합 아키텍처

점대점 통합 | 시스템마다 서로 직접 연결해 데이터를 주고받는 방식입니다.

EAI | 기업 내부의 여러 애플리케이션과 데이터를 통합하는 아키텍처와 솔루션입니다.

ESB | 서비스 버스에 시스템을 연결해 메시지 라우팅·변환·중재·보안·모니터링을 공통 처리하는 구조입니다.

API Gateway | 외부 클라이언트와 내부 서비스 사이에서 인증·라우팅·속도 제한·집계·변환을 담당하는 진입점입니다.

IoT와 엣지

IoT (Internet of Things) | 센서와 장치가 네트워크로 연결되어 데이터를 수집·교환하고 제어하는 기술입니다.

MQTT | 저대역폭·불안정한 네트워크의 장치를 위한 경량 발행·구독 메시징 프로토콜입니다.

엣지 컴퓨팅 | 데이터가 발생하는 장치나 네트워크 가장자리에서 가까운 곳에서 처리하는 방식입니다.

빅데이터와 AI

빅데이터 5V | Volume·Velocity·Variety·Veracity·Value로 대규모 데이터의 특징을 설명합니다.

머신러닝 (Machine Learning) | 데이터에서 패턴을 학습해 예측·분류 등에 활용하는 기술입니다.

딥러닝 (Deep Learning) | 여러 층의 신경망으로 복잡한 패턴을 학습하는 머신러닝 방법입니다.

데이터 레이크 (Data Lake) | 정형·반정형·비정형 원천 데이터를 원형에 가깝게 모아 저장하는 공간입니다.

분산 처리 (Distributed Processing) | 대용량 작업을 여러 컴퓨터에 나누어 병렬로 실행하는 방식입니다.

블록체인과 신뢰 기술

블록체인 | 거래 기록을 여러 노드가 공유하고 블록을 해시로 연결해 변경을 어렵게 하는 분산 원장 기술입니다.

합의 알고리즘 | 분산된 노드가 어떤 거래와 원장 상태를 정본으로 인정할지 결정하는 방식입니다.

스마트 계약 | 블록체인에 배포된 프로그램이 미리 정한 조건을 만족하면 거래나 상태 변경을 자동 실행합니다.

디지털 트윈 | 현실의 장비·공정·공간을 디지털 모델로 복제하고 실시간 데이터로 상태를 갱신하는 기술입니다.

시간 지표 공식

완료 시각을 찾은 뒤 반환=완료-도착, 대기=반환-실행, 응답=첫 실행-도착을 적용합니다.

반환시간 (Turnaround Time) | 프로세스가 도착한 순간부터 모든 실행을 마치기까지 걸린 시간입니다.

대기시간 (Waiting Time) | 프로세스가 준비 큐에서 CPU를 기다린 시간의 합입니다.

응답시간 (Response Time) | 프로세스가 도착한 뒤 처음으로 CPU를 받기까지 걸린 시간입니다.

평균 시간 | 각 프로세스에서 구한 같은 종류의 시간을 모두 더해 프로세스 수로 나눈 값입니다.

알고리즘별 시간축

FCFS | 먼저 도착한 프로세스를 먼저 실행하는 비선점형 스케줄링입니다.

SJF | 현재 준비된 작업 중 실행 시간이 가장 짧은 프로세스를 선택하는 비선점형 스케줄링입니다.

SRTF | 현재 작업의 남은 시간과 새로 도착한 작업의 실행 시간을 비교해 더 짧은 작업을 실행합니다.

Round Robin (RR) | 준비 큐의 각 프로세스에 시간 할당량만큼 CPU를 주고 미완료 작업은 큐 뒤로 보냅니다.

HRN | 대기시간을 반영한 응답률이 가장 높은 프로세스를 선택하는 비선점 방식입니다.

계산 실수 방지

도착 시간 확인 | 현재 시각까지 도착한 프로세스만 준비 큐에서 선택할 수 있습니다.

동률 처리 | 실행 시간이나 우선순위가 같을 때 적용할 도착순·큐 순서 조건을 확인합니다.

남은 실행 시간 | 선점된 프로세스는 이미 실행한 시간을 전체 실행 시간에서 빼고 다시 큐에 둡니다.

공식 검산 | 각 프로세스에서 대기시간+실행시간이 반환시간과 같은지 확인합니다.

비선점형 시간축 계산 예시

공통 계산 공식 | 완료 시각 C, 도착 A, 실행 B일 때 반환시간 T=C-A, 대기시간 W=T-B입니다.

FCFS (선착순) | 도착 순서로 P1 0~6 → P2 6~9 → P3 9~10을 실행합니다.

SJF (최단 작업 우선) | 시각 0에는 P1만 도착했으므로 P1 0~6, 이후 짧은 P3 6~7, P2 7~10입니다.

HRN (최고 응답률 우선) | 시각 6에 응답률 R=(대기시간+실행시간)/실행시간이 가장 큰 작업을 선택합니다.

선점형 시간축 계산 예시

SRTF (최단 잔여시간 우선) | 시각 1에 P2(3)가 P1의 잔여 5보다 짧아 선점합니다.

RR (시간 할당량 2) | 각 작업을 최대 2만큼 실행하고 미완료 작업은 준비 큐 뒤로 보냅니다.

응답시간·반환시간 검산 | 응답시간=첫 실행-도착, 반환시간=완료-도착입니다.

프리픽스 변환

부분 마스크는 128·192·224·240·248·252·254, 블록 크기는 256에서 부분 마스크를 뺍니다.

/24~30 빠른 표 | /24부터 마지막 옥텟의 마스크는 0·128·192·224·240·248·252 순서입니다.

호스트 비트 | IPv4 전체 32비트에서 프리픽스 길이를 뺀 값입니다.

변화하는 옥텟 | 서브넷 경계가 실제로 바뀌는 마스크의 부분 옥텟입니다.

블록 크기 | 변화하는 옥텟에서 한 서브넷이 차지하는 주소 간격입니다.

주소 범위 계산

네트워크 주소	호스트 비트를 모두 0으로 만든 해당 주소 블록의 첫 번째 주소입니다.
브로드캐스트 주소	호스트 비트를 모두 1로 만든 해당 주소 블록의 마지막 주소입니다.
사용 가능 범위	일반적인 서브넷에서 네트워크와 브로드캐스트를 제외한 주소 범위입니다.
같은 네트워크 판별	두 IP에 같은 마스크를 적용했을 때 네트워크 주소가 같으면 같은 서브넷입니다.

서브넷 분할

동일 크기 분할 (FLSM) | 모든 서브넷에 같은 수의 주소를 배정하도록 호스트 비트를 빌리는 방식입니다.

새 프리픽스 | 기존 프리픽스에 서브넷 분할에 빌린 비트 수를 더한 값입니다.

필요 호스트 수 역산 | 필요한 장비 수에 네트워크·브로드캐스트 2개를 더해 수용 가능한 최소 2의 거듭제곱을 찾습니다.

/31 점대점 링크 | IPv4 /31에는 주소가 2개이며 점대점 링크에서는 두 끝점에 각각 사용할 수 있습니다.

/32 단일 호스트 | IPv4 /32는 주소 하나를 정확히 지정하는 접두 길이입니다.

브로드캐스트 주소 계산 공식·예시

비트 계산 공식 | IPv4 네트워크 주소는 IP AND 서브넷 마스크, 브로드캐스트 주소는 네트워크 OR (32비트 마스크의 반전)입니다.

/26 마지막 옥텟 예시 | 192.168.10.77/26은 블록 크기 256-192=64로 64~127 구간에 속합니다.

/22 세 번째 옥텟 예시 | 172.16.9.130/22는 마스크 255.255.252.0으로 세 번째 옥텟이 4씩 변합니다.

브로드캐스트 예외·구분 | 255.255.255.255는 로컬 한정 브로드캐스트이고 192.168.10.127은 위 /26 서브넷의 지향 브로드캐스트입니다.

서브넷 분할 계산 공식·실기형 예시

문제 조건 확인 | 원래 대역 10.20.4.64/27을 2분할하고, 10.20.4.82가 속한 서브넷의 프리픽스와 브로드캐스트 주소를 구합니다.

빌릴 비트·새 마스크 | 2개로 분할하므로 2^s≥2인 최소 s=1을 빌려 /28로 만듭니다.

IP가 속한 구간 | 10.20.4.82의 마지막 옥텟 82는 둘째 구간 80~95에 있습니다.

최종 답안과 검산 | 정답은 프리픽스 /28과 브로드캐스트 주소 10.20.4.95입니다.

정보시스템 구축 관리

비트 표로 푸는 브로드캐스트와 수신 IP

호스트 비트 10개 | 255.255.252.0은 11111111.11111111.11111100.00000000이며 1이 22개, 0이 10개입니다.

네트워크 주소 · 호스트 0 | 35의 이진수 00100011에서 앞 6비트 001000을 유지하고 뒤 2비트를 00으로 바꾸면 00100000=32입니다.

브로드캐스트 주소 · 호스트 1 | 앞 6비트 001000을 유지한 채 뒤 2비트를 11로 만들면 00100011=35입니다. 마지막 옥텟은 11111111=255입니다.

수신 PC의 IP 구분 | 같은 /22 서브넷의 일반 호스트 범위는 192.168.32.1~192.168.35.254입니다. 각 후보 IP의 네트워크 주소를 비교합니다.

표 작성 원칙

페이지 적중 (Hit) | 요청한 페이지가 이미 프레임에 있어 추가 적재가 필요 없는 상태입니다.
페이지 부재 (Fault) | 요청한 페이지가 프레임에 없어 보조기억장치에서 가져와야 하는 상태입니다.
초기 적재와 교체 | 초기 적재는 빈 프레임을 채우는 작업이고 교체는 기존 페이지를 내보내고 새 페이지를 넣는 작업입니다.
프레임 표 | 각 참조 직후 프레임의 전체 상태를 열 단위로 기록한 표입니다.

교체 알고리즘

FIFO | 프레임에 가장 먼저 들어온 페이지를 가장 먼저 교체합니다.

LRU | 현재 시점에서 가장 오랫동안 참조되지 않은 페이지를 교체합니다.

LFU | 누적 참조 횟수가 가장 적은 페이지를 교체합니다.

OPT | 앞으로 가장 늦게 다시 사용되거나 다시 사용되지 않는 페이지를 교체합니다.

NUR/NRU (최근 미사용) | 참조 비트 R과 변경 비트 M으로 페이지를 분류해 교체 대상을 고릅니다.

SCR (2차 기회·Second Chance) | FIFO 순서에서 참조 비트가 1인 페이지에 한 번 더 기회를 줍니다.

계산 검산

부재 횟수 검산 | 전체 참조 횟수에서 페이지 적중 횟수를 빼면 페이지 부재 횟수가 됩니다.

교체 횟수 검산 | 빈 프레임을 모두 채운 이후 발생한 부재 중 기존 페이지를 내보낸 횟수입니다.

Belady 이상 현상 | FIFO에서는 프레임 수를 늘렸는데도 페이지 부재가 오히려 증가할 수 있는 현상입니다.

조건 우선 적용 | 초기 프레임 상태, 동률 처리, 참조 횟수 초기화 방식은 문제에서 제시한 조건을 따릅니다.

주소·제어 프로토콜

ARP | 같은 네트워크에서 알려진 IP 주소에 대응하는 MAC 주소를 알아내는 프로토콜입니다.

RARP | MAC 주소를 이용해 자신의 IP 주소를 알아내던 역방향 주소 해석 프로토콜입니다.

ICMP | IP 패킷 전달 중 오류와 진단 정보를 전달하는 제어용 프로토콜입니다.

IGMP | IPv4 호스트와 라우터 사이에서 멀티캐스트 그룹 가입·탈퇴 정보를 교환합니다.

NAT | 네트워크 경계에서 IP 주소를 다른 주소로 변환하는 기술입니다.

HDLC와 오류 제어

HDLC 프레임 | I 프레임은 정보 전송, S 프레임은 흐름·오류 제어, U 프레임은 번호 없는 링크 관리에 사용됩니다.

ABM (비동기 균형 모드) | HDLC에서 두 단말이 대등하게 명령과 응답을 주고받는 동작 모드입니다.

ARM (비동기 응답 모드) | HDLC에서 종국이 주국의 허가를 받아 전송하는 불균형 동작 모드입니다.

패리티 검사 (Parity) | 데이터 비트의 1 개수가 짝수 또는 홀수가 되도록 검사 비트를 덧붙이는 오류 검출 방식입니다.

CRC | 생성 다항식으로 데이터를 나눈 나머지를 검사값으로 붙이는 오류 검출 방식입니다.

해밍 코드 (Hamming Code) | 검사 비트를 배치해 오류 위치를 찾아 단일 비트 오류를 정정할 수 있는 부호입니다.

FEC (전진 오류 정정) | 재전송 없이 수신 측이 추가된 오류 정정 정보를 이용해 오류를 복구하는 방식입니다.

BEC (후진 오류 정정) | 수신 측이 오류를 검출해 송신 측에 재전송을 요청하는 방식입니다.

공격 유형

Session Hijacking (세션 하이재킹) | 인증된 사용자의 세션을 탈취하여 다시 비밀번호를 확인하지 않고 권한을 악용합니다.

Watering Hole (워터링 홀) | 공격 대상이 자주 방문하는 사이트를 감염시켜 그 사이트의 방문자를 공격합니다.

스케어웨어 (Scareware) | 가짜 감염 경고로 공포심을 유발하여 결제나 악성 프로그램 설치를 유도합니다.

이블 트윈 (Evil Twin) | 정상 무선 AP처럼 위장한 가짜 AP로 접속을 유도하여 통신이나 정보를 노립니다.

구별: 스케어웨어는 가짜 경고로 속이는 방식, 랜섬웨어는 실제 암호화·접근 차단 후 금전을 요구하는 방식입니다.

인증과 관리

OTP (일회용 비밀번호) | 한 번 사용하거나 짧은 시간만 유효한 비밀번호로 인증값의 재사용을 줄입니다.

OAuth (권한 위임) | 비밀번호를 제3자 서비스에 넘기지 않고 토큰으로 자원 접근 권한을 위임합니다.

SSO (통합 인증) | 한 번 인증한 사용자가 여러 관련 서비스에 다시 로그인하지 않고 접근합니다.

ISMS (정보보호 관리체계) | 조직의 정보보호 정책과 위험·보호대책을 지속적으로 운영하고 점검하는 체계입니다.

구별: OAuth는 권한 위임 표준이며 그 자체가 로그인 인증 표준은 아닙니다. OpenID Connect는 OAuth 2.0 위에 인증 계층을 추가합니다.

악성코드 종류와 구별

바이러스 (Virus) | 다른 프로그램이나 파일에 붙어 복제하며 감염된 숙주가 실행될 때 활성화됩니다.

웜 (Worm) | 숙주 프로그램에 붙지 않고 독립적으로 복제되어 다른 시스템으로 전파됩니다.

트로이목마 (Trojan Horse) | 정상 프로그램으로 위장해 숨은 악성 기능을 실행하며 자체 복제가 핵심은 아닙니다.

랜섬웨어 (Ransomware) | 파일 암호화나 접근 차단으로 피해를 주고 복구 대가로 금전을 요구합니다.

스파이웨어 (Spyware) | 사용자 모르게 활동과 정보를 수집해 외부로 전달하는 감시 소프트웨어입니다.

루트킷 (Rootkit) | 공격자의 파일이나 프로세스 등을 숨겨 보안 검사에서 발견하기 어렵게 합니다.

키로거 (Keylogger) | 키보드 입력을 기록하는 도구로 악용하면 비밀번호 등 입력 정보를 훔칩니다.

백도어 (Backdoor) | 정상적인 인증이나 보안 검사를 우회해 시스템에 접근하는 숨겨진 경로입니다.

논리 폭탄 (Logic Bomb) | 지정한 날짜나 사건 등 조건이 충족될 때 숨겨 둔 악성 동작을 실행합니다.

구별: 바이러스·웜은 복제·전파 방식, 트로이목마는 위장, 랜섬웨어는 피해 행동에 주목한 구분입니다. 한 악성코드가 여러 특성을 함께 가질 수 있습니다.

IPC

Message Queue | 프로세스가 메시지를 큐에 넣고 다른 프로세스가 꺼내는 비동기 통신 방식입니다.
공유 메모리 (Shared Memory) | 여러 프로세스가 같은 메모리 영역에 접근해 데이터를 교환하는 IPC 방식입니다.
소켓 (Socket) | 같은 호스트 또는 네트워크의 통신 끝점 사이에서 데이터를 주고받는 IPC 방식입니다.
Semaphore | 여러 프로세스가 공유 자원에 접근하는 순서를 제어하는 동기화 도구입니다.

재해복구와 신기술

BIA (업무 영향 분석) | 업무가 중단되었을 때 발생하는 손실과 중요도·복구 우선순위를 분석합니다.

BCP (업무 연속성 계획) | 재해나 장애 중에도 핵심 업무를 지속하거나 빠르게 재개하기 위한 종합 계획입니다.

RTO (목표 복구 시간) | 장애 발생 후 서비스를 복구하기까지 허용하는 목표 시간입니다.

RPO (목표 복구 시점) | 복구할 데이터가 장애 시점에서 얼마나 이전까지여도 허용되는지를 나타냅니다.

DRP (재해 복구 계획) | 재해 뒤 정보시스템과 데이터를 복구하기 위한 절차와 책임을 정한 계획입니다.

DRS (재해 복구 시스템) | 재해 발생 시 서비스를 복구하기 위해 준비한 대체 시스템과 인프라입니다.

SDDC | 서버·네트워크·스토리지와 데이터센터 운영을 소프트웨어로 정의하고 자동화하는 데이터센터입니다.

HDFS | 대용량 파일을 여러 노드의 블록으로 나누어 분산 저장하는 파일 시스템입니다.

MapReduce | 대용량 데이터를 나누어 병렬 처리하고 중간 결과를 집계하는 분산 처리 모델입니다.

VLAN·메시·애드혹·NFC

VLAN (가상 LAN) | 물리 위치와 별개로 논리적 LAN과 브로드캐스트 영역을 나눕니다.

Mesh (메시 네트워크) | 노드 간 여러 연결과 경로를 활용하는 망 구조입니다.

Ad-hoc (애드혹 네트워크) | 고정 기반 시설 없이 기기들이 임시로 구성하는 망입니다.

NFC (근거리 무선 통신) | 매우 가까운 거리에서 기기 간 정보를 교환하는 무선 기술입니다.

반입·배치·교체 전략

반입 (Fetch) | 보조기억장치의 자료를 주기억장치로 가져올 시점을 정합니다.

배치 (Placement) | 새로 적재할 자료를 메모리의 어느 영역에 놓을지 정합니다.

교체 (Replacement) | 공간이 부족할 때 내보낼 페이지나 자료를 정합니다.

정보시스템 구축 관리

최초·최적·최악 적합 계산

최초 적합 (First Fit) | 주소 순서대로 탐색해 처음으로 충분한 빈 영역에 배치합니다.

최적 적합 (Best Fit) | 수용 가능한 빈 영역 중 크기가 가장 작은 곳을 선택합니다.

최악 적합 (Worst Fit) | 수용 가능한 빈 영역 중 크기가 가장 큰 곳을 선택합니다.

지역성·워킹 셋·PFF

시간 지역성 (Temporal Locality) | 최근 참조한 자료나 명령을 가까운 미래에 다시 참조하는 성향입니다.

공간 지역성 (Spatial Locality) | 참조한 주소 주변의 자료나 명령을 이어서 참조하는 성향입니다.

워킹 셋 (Working Set) | 일정한 최근 구간 동안 참조한 페이지들의 집합입니다.

스래싱 (Thrashing) | 페이지 교체가 과도해 유용한 실행보다 메모리 입출력에 시간을 소비합니다.

PFF (페이지 부재 빈도 방식) | 페이지 부재율에 따라 프레임 배분을 조정합니다.

프리페이징 (Prepaging) | 앞으로 필요할 것으로 예상하는 페이지를 미리 적재합니다.

정보시스템 구축 관리

메모리 분할과 단편화

페이징 (Paging) | 논리 메모리를 일정 크기 페이지로, 물리 메모리를 같은 크기 프레임으로 나눕니다.

세그먼테이션 (Segmentation) | 코드·데이터·스택 등 논리적 의미를 가진 가변 크기 단위로 나눕니다.

내부 단편화 (Internal Fragmentation) | 이미 할당한 영역 안에서 사용하지 못하는 공간입니다.

외부 단편화 (External Fragmentation) | 빈 공간이 여러 작은 조각으로 흩어져 연속 요청을 수용하지 못합니다.

스토리지 연결 방식

DAS (직접 연결 스토리지) | 서버나 호스트에 저장장치를 직접 연결합니다.

NAS (네트워크 연결 스토리지) | 네트워크를 통해 파일 단위의 저장 서비스를 제공합니다.

SAN (스토리지 영역 네트워크) | 저장장치와 서버 사이에 블록 단위 접근을 제공하는 전용 네트워크입니다.

TCP 윈도우와 혼잡 제어

수신 윈도우 (rwnd) | 수신자가 받아들일 수 있는 데이터량을 송신자에게 알립니다.

혼잡 윈도우 (cwnd) | 송신자가 네트워크 혼잡 상태를 반영해 전송량을 제한합니다.

느린 시작 (Slow Start) | ACK가 도착하면 윈도우를 늘려 RTT 기준 대략 지수적으로 증가시킵니다.

혼잡 회피 (Congestion Avoidance) | 임계값 이후 전송량을 대략 선형적으로 늘립니다.

빠른 재전송 (Fast Retransmit) | 중복 ACK 등으로 손실을 추정해 타임아웃 전에 재전송합니다.

회선·데이터그램·가상 회선

회선 교환 (Circuit Switching) | 연결 설정 후 통신 경로의 자원을 확보해 사용합니다.

데이터그램 (Datagram) | 각 패킷이 목적지 정보를 가지고 독립적으로 전달됩니다.

가상 회선 (Virtual Circuit) | 논리적인 연결을 설정하고 그 연결을 따라 패킷을 전달합니다.

분산 데이터 처리 구성요소

Hadoop (하둡) | 대규모 데이터의 분산 저장·처리를 위한 소프트웨어 생태계입니다.

HDFS (하둡 분산 파일 시스템) | 파일을 블록으로 나누어 여러 노드에 저장합니다.

MapReduce (맵리듀스) | Map과 Reduce 단계로 큰 작업을 분산·집계하는 처리 모델입니다.

Hive (하이브) | 분산 저장 데이터에 SQL 계열 질의를 제공하는 데이터 웨어하우스 도구입니다.

ZooKeeper (주키퍼) | 분산 시스템의 설정·이름·동기화·조정을 지원합니다.

온톨로지·시맨틱 웹·LOD

온톨로지 (Ontology) | 특정 영역의 개념과 관계를 명시적으로 표현한 지식 모델입니다.

시맨틱 웹 (Semantic Web) | 웹 데이터의 의미를 기계가 처리할 수 있도록 표현하는 접근입니다.

LOD (연결된 개방형 데이터) | 개방된 데이터를 URI와 링크로 연결해 활용하도록 하는 방식입니다.

매시업 (Mashup) | 여러 데이터나 서비스를 조합해 새로운 서비스를 만듭니다.

대칭키 알고리즘의 특징

DES (데스) | 64비트 블록과 유효 56비트 키를 사용하는 구형 대칭키 암호입니다.

AES (고급 암호 표준) | 블록은 128비트이며 키는 128·192·256비트입니다.

SEED (시드) | 기본 SEED 규격은 128비트 블록과 128비트 키를 사용합니다.

ARIA (아리아) | 128비트 블록과 128·192·256비트 키를 지원하는 국산 블록 암호입니다.

RC4 (알시포) | 키스트림과 평문을 XOR하는 구형 스트림 암호입니다.

비대칭 암호와 키 합의

RSA (알에스에이) | 소인수분해의 어려움과 관련된 공개키 암호 체계입니다.

Diffie-Hellman (디피-헬만) | 공개된 통신으로 공통 비밀값을 합의하는 키 합의 방식입니다.

ECC (타원곡선 암호) | 타원곡선상의 이산대수 문제를 활용하는 공개키 기술 계열입니다.

ElGamal (엘가말) | 이산대수 문제를 기반으로 하는 공개키 암호 체계입니다.

해시 함수의 출력 길이

MD5 (엠디파이브) | 128비트의 해시값을 출력하는 구형 해시 함수입니다.

SHA-1 (샤 원) | 160비트의 해시값을 출력하는 구형 해시 함수입니다.

SHA-256 (샤 이백오십육) | SHA-2 계열이며 256비트 해시를 출력합니다.

SHA-384 (샤 삼백팔십사) | SHA-2 계열이며 384비트 해시를 출력합니다.

SHA-512 (샤 오백십이) | SHA-2 계열이며 512비트 해시를 출력합니다.

HAS-160 (해즈 백육십) | 160비트 출력을 사용하는 국내 개발 해시 함수입니다.

벨-라파듈라·비바·클라크-윌슨

Bell-LaPadula (벨-라파듈라) | 기밀성을 위해 보안 등급 간 정보 흐름을 제한합니다.

Biba (비바) | 무결성을 위해 신뢰도가 낮은 정보가 높은 정보를 오염시키지 못하게 합니다.

Clark-Wilson (클라크-윌슨) | 검증된 거래 절차와 직무 분리로 무결성을 유지합니다.

통합 보안·무선 보안·유출 방지

WIPS (무선 침입 방지) | 비인가 무선 접속과 무선 환경의 침입을 탐지·차단합니다.

UTM (통합 위협 관리) | 방화벽·침입 방지·VPN 등 여러 보안 기능을 통합합니다.

ESM (통합 보안 관리) | 여러 보안 시스템의 상태·이벤트를 중앙에서 관리합니다.

SIEM (보안 정보·이벤트 관리) | 다양한 로그와 이벤트를 수집·상관 분석합니다.

DLP (데이터 유출 방지) | 중요 정보의 외부 반출을 탐지하거나 통제합니다.

CVE·CWE·CVSS·CC

CVE (공개 취약점 식별 체계) | 공개된 개별 취약점을 공통 식별자로 구별합니다.

CWE (공통 약점 목록) | 취약점의 원인이 되는 소프트웨어·하드웨어 약점 유형을 분류합니다.

CVSS (공통 취약점 점수 체계) | 취약점의 기술적 심각도를 일관된 척도로 평가합니다.

CC (공통 평가 기준) | 정보보호 제품의 보안 기능과 보증을 평가하는 국제 공통 기준입니다.

IPSec의 AH·ESP·IKE와 모드

AH (인증 헤더) | IP 패킷의 무결성과 데이터 출처 인증을 제공합니다.

ESP (보안 페이로드 캡슐화) | 암호화에 의한 기밀성과 설정에 따른 무결성·인증을 제공합니다.

IKE (인터넷 키 교환) | 상대를 인증하고 키와 보안 연결 설정을 협상합니다.

전송 모드 (Transport Mode) | 주로 원래 IP 패킷의 상위 계층 데이터 부분을 보호합니다.

터널 모드 (Tunnel Mode) | 원래 IP 패킷을 내부에 넣고 새 IP 패킷으로 전달합니다.

TLS 메시지의 역할과 버전

Handshake (핸드셰이크) | 통신 상대와 보안 매개변수·키를 협상합니다.

Record (레코드) | 메시지를 레코드 단위로 구성해 보호하고 전달합니다.

Alert (얼럿) | 오류나 연결 종료 등 프로토콜 상태를 알립니다.

ChangeCipherSpec (암호 규격 변경) | TLS 1.2 등에서 합의한 암호 상태로 전환함을 알립니다.

플러딩·반사·패킷 처리 공격

SYN Flooding (신 플러딩) | 다량의 연결 요청으로 TCP 연결 대기 자원을 소모시킵니다.

UDP Flooding (유디피 플러딩) | 다량의 UDP 패킷으로 회선이나 처리 자원을 소모시킵니다.

DRDoS (분산 반사 서비스 거부) | 여러 반사 서버의 응답이 피해자에게 모이도록 유도합니다.

Ping of Death (죽음의 핑) | 비정상적으로 큰 IP 데이터그램의 처리 취약점을 이용하는 고전적 공격입니다.

Teardrop (티어드롭) | 잘못 겹치는 IP 조각 등으로 재조립 취약점을 악용합니다.

느린 HTTP와 해시 자원 고갈

Slowloris (느린 HTTP 헤더) | HTTP 헤더를 끝내지 않아 서버가 연결을 기다리게 합니다.
RUDY (느린 HTTP 본문) | 큰 요청 본문을 예고하고 매우 천천히 보내 연결을 유지합니다.
Slow HTTP Read (느린 응답 읽기) | 클라이언트가 응답을 지나치게 천천히 읽어 서버 자원을 붙잡습니다.
Hash DoS (해시 서비스 거부) | 다수의 해시 충돌로 자료 처리의 계산량을 늘립니다.

표적과 유입 경로로 보는 공격

스피어 피싱 (Spear Phishing) | 특정 개인이나 조직을 겨냥해 맞춤형으로 속이는 피싱입니다.

스미싱 (Smishing) | SMS 등 문자 메시지를 이용한 피싱입니다.

큐싱 (Quishing) | QR 코드를 이용해 가짜 접속 경로로 유도하는 피싱입니다.

공급망 공격 (Supply Chain Attack) | 개발·업데이트·납품 등 신뢰하는 공급 경로를 악용합니다.

APT (지능형 지속 위협) | 특정 대상을 상대로 장기간 여러 수단을 조합하는 위협입니다.

제로데이 공격 (Zero-day Attack) | 충분한 패치·대응이 마련되지 않은 취약점을 악용합니다.

사전·대입·스터핑·레인보우 테이블

사전 공격 (Dictionary Attack)	사용할 가능성이 높은 단어·후보 비밀번호를 시도합니다.
무차별 대입 (Brute Force)	정한 탐색 공간의 가능한 조합을 폭넓게 시도합니다.
하이브리드 공격 (Hybrid Attack)	사전 후보에 숫자·기호 등의 변형을 결합합니다.
크리덴셜 스터핑 (Credential Stuffing)	다른 서비스에서 유출된 ID·비밀번호 조합을 재사용합니다.
레인보우 테이블 (Rainbow Table)	사전 계산한 해시 연결 정보를 이용해 후보 비밀번호를 찾습니다.

블루프린팅·재킹·스나핑·버깅

블루프린팅 (BluePrinting) | 블루투스 기기의 특성을 수집해 종류·모델 등을 식별합니다.

블루재킹 (BlueJacking) | 주변 블루투스 기기에 원치 않는 메시지 등을 보냅니다.

블루스나핑 (BlueSnarfing) | 블루투스 취약점을 통해 기기의 데이터를 무단으로 가져갑니다.

블루버깅 (BlueBugging) | 취약점을 악용해 기기의 기능에 무단 접근·제어합니다.

XSS의 반사·저장·DOM 유형

Reflected XSS (반사형) | 요청에 포함된 입력이 응답에 안전하지 않게 반영됩니다.

Stored XSS (저장형) | 저장된 악성 입력이 다른 사용자의 화면에서 실행됩니다.

DOM-based XSS (DOM 기반) | 브라우저 코드가 신뢰하지 않은 값을 위험한 DOM 조작에 사용합니다.

정보시스템 구축 관리

SQL 삽입의 관찰 방식

Error-based (오류 기반) | DB의 오류 메시지에 드러나는 정보로 질의 동작을 추론합니다.

Union-based (UNION 기반) | 원래 조회 결과에 다른 조회 결과를 결합하는 취약점 유형입니다.

Blind (블라인드) | 직접 결과가 없어도 응답의 참·거짓 차이나 지연으로 정보를 추론합니다.

메모리 노출·중간자·부채널

Heartbleed (하트블리드) | 일부 OpenSSL 버전의 TLS Heartbeat 처리에서 메모리가 노출된 취약점입니다.

Spectre (스펙터) | 추측 실행과 부채널을 악용하는 CPU 취약점 계열입니다.

Meltdown (멜트다운) | 일부 CPU의 비순차 실행·권한 처리 특성과 부채널을 악용한 취약점입니다.

MITM (중간자 공격) | 통신 중간에서 당사자를 속여 내용을 가로채거나 바꾸는 공격입니다.

부채널 (Side Channel) | 처리 시간·전력·캐시 등 부수적 정보로 비밀을 추론합니다.

정보시스템 구축 관리

좀비 기기·C2·봇넷

좀비 기기 (Zombie Device) | 악성코드에 감염되어 외부의 명령에 따라 동작하는 기기입니다.

C2·C&C (명령·제어) | 감염 기기에 명령을 전달하고 결과를 수집하는 체계입니다.

봇넷 (Botnet) | 공동으로 제어되는 감염 기기들의 네트워크입니다.

탬퍼 프루핑과 프로그램 변조 탐지

변조 탐지·대응 | 코드나 데이터의 변경 여부를 검사하고 정책에 따라 경고·실행 중지 등으로 대응합니다.

난독화와의 차이 | 난독화는 분석·이해를 어렵게 하고 탬퍼 프루핑은 무단 변경의 탐지·저항에 초점을 둡니다.

검증 기준의 신뢰 | 서명된 배포 정보나 보호된 기준처럼 비교 대상의 신뢰성도 확인해야 합니다.

심볼릭 링크 공격과 경로 검증

심볼릭 링크 (기능) | 다른 경로를 가리키는 별도 파일이며 최종 대상 파일을 복제하지 않습니다.

공격 성립 상황 | 공격자가 조작한 링크를 더 높은 권한의 프로그램이 안전 확인 없이 따라갈 때 문제가 됩니다.

방어 관점 | 신뢰할 수 있는 디렉터리, 최소 권한, 안전한 파일 열기와 링크 처리 정책을 함께 적용합니다.

정보시스템 구축 관리

ATM의 고정 길이 셀과 가상 연결

Cell (고정 길이 셀)	하나의 ATM 셀은 헤더 5바이트와 페이로드 48바이트를 합한 53바이트입니다.
가상 경로·가상 채널	헤더의 VPI·VCI 같은 식별자로 가상 연결을 구분합니다.
패킷·회선과 구별	ATM은 고정 셀 기반이고 일반적인 IP 데이터그램은 가변 길이 패킷 기반입니다.

정보시스템 구축 관리

L2TP 터널링과 IPsec 암호화 구분

L2TP (계층 2 터널링) | PPP 프레임을 캡슐화해 터널 양끝 사이에서 전달합니다.

IPsec (보안 보호) | IP 통신에 인증·무결성·기밀성 등의 보안 기능을 제공하는 프로토콜 집합입니다.

VPN·SSH와 구별 | VPN은 가상 사설망이라는 구성·용도이고 L2TP는 구현에 사용되는 프로토콜 중 하나입니다.

chmod의 8진수 권한 계산

한 자리의 비트 합 | rwx는 4+2+1=7, r-x는 4+1=5, r--는 4입니다.

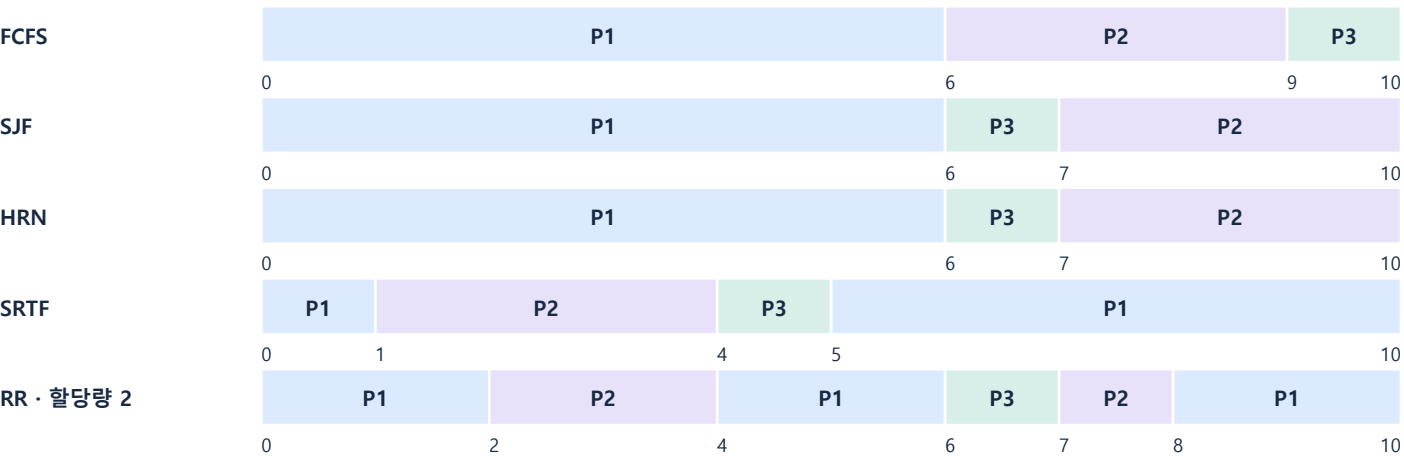
754의 사용자 구분 | 소유자 7, 그룹 5, 기타 4에 대응합니다.

문자 표현에서 숫자로 | rw-r-----라면 소유자 6, 그룹 4, 기타 0으로 읽습니다.

계산 예제 - CPU 스케줄링

반환 T = 완료 C - 도착 A | 대기 W = T - 실행 B | 응답 R = 첫 실행 - A

프로세스	도착 A	실행 B
P1	0	6
P2	1	3
P3	3	1



알고리즘	P1 / P2 / P3 대기 W	평균 W	평균 T
FCFS	0 / 5 / 6	3.67	7.00
SJF	0 / 6 / 3	3.00	6.33
HRN	0 / 6 / 3	3.00	6.33
SRTF	4 / 0 / 1	1.67	5.00
RR · 할당량 2	4 / 4 / 3	3.67	7.00

시간 단위 ms. 완료 시각은 해당 프로세스의 마지막 실행 끝이며, 대기시간에는 CPU에서 실행한 시간을 포함하지 않습니다.

FCFS: 도착 순서대로 실행합니다.

SJF: 시각 6에 도착해 있는 P2(3)와 P3(1) 중 짧은 P3를 먼저 선택합니다.

HRN: 시각 6의 응답률: P2=(5+3)/3=8/3, P3=(3+1)/1=4 → P3 선택.

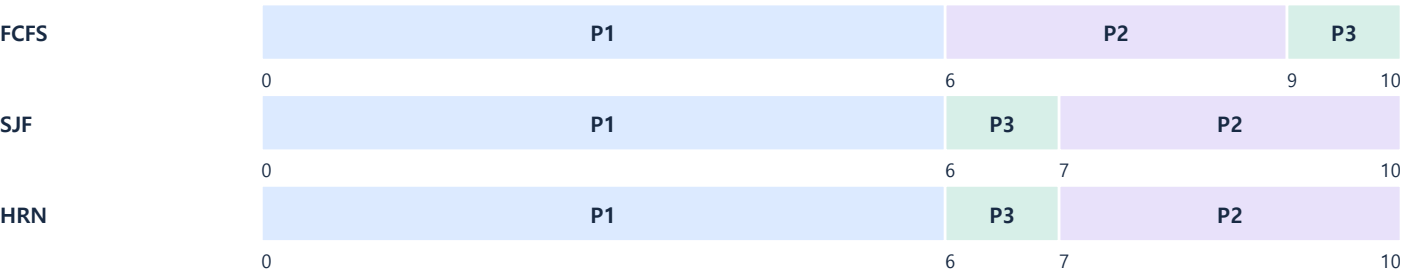
SRTF: 시각 1에 P2가 P1을 선점합니다. 시각 3에는 P2·P3의 잔여시간이 같아 현재 작업 P2를 유지한다고 가정합니다.

RR · 할당량 2: 실행 중 도착한 작업은 준비 큐에 넣고, 할당량 후 미완료 작업은 큐 맨 뒤로 보냅니다.

계산 예제 - CPU 스케줄링

반환 T = 완료 C - 도착 A | 대기 W = T - 실행 B | 응답 R = 첫 실행 - A

프로세스	도착 A	실행 B
P1	0	6
P2	1	3
P3	3	1



알고리즘	P1 / P2 / P3 대기 W	평균 W	평균 T
FCFS	0 / 5 / 6	3.67	7.00
SJF	0 / 6 / 3	3.00	6.33
HRN	0 / 6 / 3	3.00	6.33

시간 단위 ms. 완료 시각은 해당 프로세스의 마지막 실행 끝이며, 대기시간에는 CPU에서 실행한 시간을 포함하지 않습니다.

FCFS: 도착 순서대로 실행합니다.

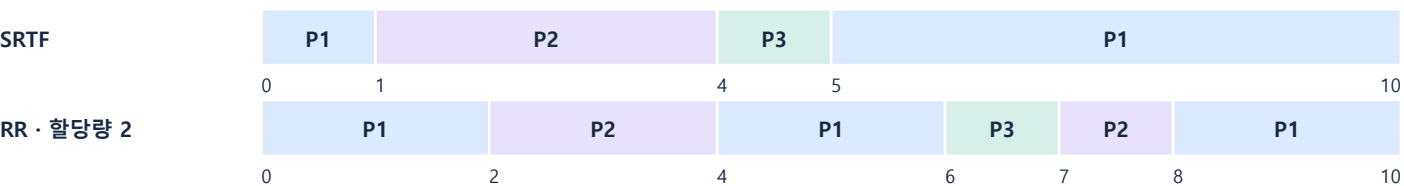
SJF: 시각 6에 도착해 있는 P2(3)와 P3(1) 중 짧은 P3를 먼저 선택합니다.

HRN: 시각 6의 응답률: P2=(5+3)/3=8/3, P3=(3+1)/1=4 → P3 선택.

계산 예제 - CPU 스케줄링

반환 T = 완료 C - 도착 A | 대기 W = T - 실행 B | 응답 R = 첫 실행 - A

프로세스	도착 A	실행 B
P1	0	6
P2	1	3
P3	3	1



알고리즘	P1 / P2 / P3 대기 W	평균 W	평균 T
SRTF	4 / 0 / 1	1.67	5.00
RR · 할당량 2	4 / 4 / 3	3.67	7.00

시간 단위 ms. 완료 시각은 해당 프로세스의 마지막 실행 끝이며, 대기시간에는 CPU에서 실행한 시간을 포함하지 않습니다.

SRTF: 시각 1에 P2가 P1을 선점합니다. 시각 3에는 P2·P3의 잔여시간이 같아 현재 작업 P2를 유지한다고 가정합니다.

RR · 할당량 2: 실행 중 도착한 작업은 준비 큐에 넣고, 할당량 후 미완료 작업은 큐 맨 뒤로 보냅니다.

계산 예제 - 네트워크·브로드캐스트 주소

호스트 비트 h = 32 - 프리픽스 | 전체 주소 = 2^h | 일반적인 호스트 수 = 2^h - 2

블록 크기 = 256 - 변화 옥텟의 마스크. IP가 들어가는 블록의 시작은 네트워크, 끝은 브로드캐스트입니다.

입력 / 마스크	네트워크	브로드캐스트	사용 가능한 주소
192.168.10.77/26 **255.255.255.192**	192.168.10.64	192.168.10.127	192.168.10.65 ~ 192.168.10.126
172.16.9.130/22 **255.255.252.0**	172.16.8.0	172.16.11.255	172.16.8.1 ~ 172.16.11.254

77/26: 블록 64 → 64~127 → 네트워크 .64, 브로드캐스트 .127. 9/22: 세 번째 옥텟 블록 4 → 8~11 → 네트워크 172.16.8.0, 브로드캐스트 172.16.11.255.

주소 검산: 네트워크는 IP AND 마스크. 브로드캐스트는 호스트 비트를 모두 1로 합니다. /31 점대점 링크에는 일반적인 2^h-2 공식을 적용하지 않습니다.

계산 예제 - 서브넷 분할

같은 크기 2^n개로 분할: 기존 프리픽스에 n을 더합니다. /27을 2분할하면 /28, 블록 16주소, 일반 호스트 14개입니다.

10.20.4.64/27을 2분할	브로드캐스트	사용 가능 범위
10.20.4.64/28	10.20.4.79	10.20.4.65 ~ 10.20.4.78
10.20.4.80/28	10.20.4.95	10.20.4.81 ~ 10.20.4.94

10.20.4.82는 둘째 블록 80~95에 포함됩니다. 정답: /28, 브로드캐스트 10.20.4.95.

계산 예제 - 페이지 교체

빈 프레임 3개, 참조 순서 1 2 3 1 4 2 5

방식 / 참조	1	2	3	1	4	2	5
FIFO 프레임	1/-/-	1/2/-	1/2/3	1/2/3	4/2/3	4/2/3	4/5/3
FIFO 부재	O	O	O	-	O	-	O
LRU 프레임	1/-/-	1/2/-	1/2/3	1/2/3	1/4/3	1/4/2	5/4/2
LRU 부재	O	O	O	-	O	O	O

초기 적재도 부재에 포함: FIFO 5회, LRU 6회. FIFO는 적재 순서, LRU는 최근 사용 시점을 기준으로 교체합니다.